Colt: Handling Traffic Bursts using Piecewise Linear Adaptive Traffic Engineering

Abd AlRhman AlQiam
Purdue University

Danish Athar
Purdue University

Ngoc Do
Meta

Ying Zhang
Meta

Sanjay G. Rao
Purdue University

Bruno Ribeiro
Purdue University

Mohit Tawarmalani
Purdue University

Abstract

Centralized wide area network (WAN) traffic engineering (TE) schemes operate at coarse time intervals. This limits their responsiveness to short timescale traffic variations. In this paper, we present Colt, a system to quickly handle traffic variations during a TE cycle. We make three contributions. First, we explore a new design point where ingress routers use a piece-wise linear policy to dynamically adjust how traffic is split across paths during a TE cycle relying solely on locally available information. Second, we formulate an optimization model for Colt's TE controller that centrally coordinates each router's policy so as to meet desired network-wide objectives. Third, we present a novel neural architecture based on an alternating optimization framework that efficiently tackles the non-convexity of Colt's problem, and performs robustly under prediction error. Our approach is different from previous neural TE schemes in the richness of policies it generates and in optimizing performance over multiple future scenarios in each TE cycle. We evaluate Colt with a multi-week trace from a large private WAN where traffic matrices (TMs) are collected at finer-time granularities. Our results show that Colt bridges 74% of the gap between the best optimization-based non-adaptive routing approach and a fully adaptive oracle approach. Further, Colt's neural models significantly outperform state-of-the-art neural TE schemes including DOTE, FIGRET, Hattrick and RedTE.

1 Introduction

Cloud and Internet Service Providers (ISPs) commit to stringent Service-Level Objectives (SLOs) for their Wide Area Networks (WANs), which require that connectivity and provisioned bandwidth between a pair of sites must hold for a specified fraction of time, ranging from 99% (2 nines) to 99.99% (4 nines) [25]. Over the last decade, these providers [16, 24, 26] have gravitated towards centralized approaches for managing their WAN traffic. Such centralized approaches allow providers to optimize traffic for desired objectives such as maximizing flow and minimizing link utilizations [4, 16, 24, 26]. While centralized approaches offer the potential to judiciously route traffic by leveraging network-wide views, they unfortunately operate over coarse time-scales (several tens of seconds to a few minutes). This is because (i) routes need to be re-optimized for new traffic matrices, and (ii) newly computed routes need to be pushed to the routers [33, 44]. Consequently, routes computed by the controller are fixed throughout the TE cycle, making these centralized approaches unresponsive to fluctuations in traffic patterns during a TE cycle.

Our characterizations of traffic data collected from a large private WAN (PrivWAN) at 10-second granularity indicate that WAN traffic does show significant variations even within a minute (§2.1). For example, for the most active source-destination pair (i.e., the aggregate traffic between two nodes), the median and 90th percentile of the peak-to-mean ratio computed across one-minute bins are 1.21 and 1.46, respectively, with similar trends for other top pairs.

Contributions. In this paper, we present **Colt**[1], a system designed to quickly handle short-term variations in traffic *during a TE cycle*. Colt is motivated by the fact that even the best non-adaptive routing strategy can perform sub-optimally owing to traffic variations within a TE cycle. Instead, in Colt, (i) ingress routers *self-tune* how traffic is split across paths during a TE cycle while only leveraging locally available information; and (ii) the parameters of each router's self-tuning mechanism are centrally coordinated so as to meet desired network-wide objectives.

The space of locally tunable adaptation mechanisms is large. As a first step, Colt considers a simple policy where the traffic corresponding to a source and destination pair routed along a particular path is a *piecewise linear* function of the traffic of that pair. Intuitively, Colt views each pair's traffic as composed of (i) normal traffic routed with a default set of traffic splits; and (ii) excess traffic routed with a different set of traffic splits. Colt seeks to route normal traffic using non-shared paths to the extent possible leaving slack capacity

[1] Centrally orchestrated locally tunable

on shared paths with most flexibility to handle bursty traffic.

We present a novel optimization model for Colt's TE controller. The key distinction of this model relative to traditional TE controllers is that for each pair it produces (i) multiple split ratios (corresponding to normal and bursty traffic); and (ii) a threshold that determines normal traffic for that pair. Solving the model presents three difficulties. First, the formulation is non-convex unlike many traditional TE problems that are linear programs. Second, the model seeks to identify parameters that ensure good performance over a *set of matrices* in the TE cycle rather than a single matrix. Third, the model must continue to work well under imperfect predictions.

To tackle these issues, we have designed a custom neural network as part of Colt. Our use of neural architectures is motivated by their ability to better cope with prediction error [7,39,42], and to support more flexible policies that optimize performance after accounting for congestion effects [6]. Existing neural TE schemes do not support Colt's rich policies and focus on optimizing performance for a single TM rather than across a set of TMs. Colt addresses this through (i) a novel neural architecture aligned with an alternating optimization approach to discover lucrative solutions to the model described above; and (ii) the use of appropriate neural modules (e.g., set transformer [35, 51, 57]) to optimize performance across scenarios between consecutive TE controller decisions.

We evaluate Colt using a three-week dataset from PrivWAN, which includes multiple traffic matrices collected each minute, and with GEANT, the largest publicly available traffic dataset. Our results show that

• Colt outperforms state-of-the-art optimization-based and neural central TE schemes, significantly bridging the gap in total traffic delivered between these schemes and a fully adaptive oracle approach that knows each future matrix. In particular, on PrivWAN, Colt bridges 74% of the gap between the best optimization-based non-adaptive approach and the oracle, and 83% of the gap between Hattrick [6] (a recent neural approach) and the oracle, while also significantly outperforming DOTE and FIGRET.

• Relative to RedTE, a distributed neural TE scheme, Colt scales better to larger topologies, allows more frequent centralized orchestration, and employs more interpretable local adaptation policies. Further, Colt can handle datasets with topology changes unlike DOTE, FIGRET and RedTE.

• Ablation experiments demonstrate the importance of Colt's locally adaptive policies, and the benefits of its architecture in optimizing for multiple TMs.

Overall, these results show the feasibility and promise of Colt's approach.

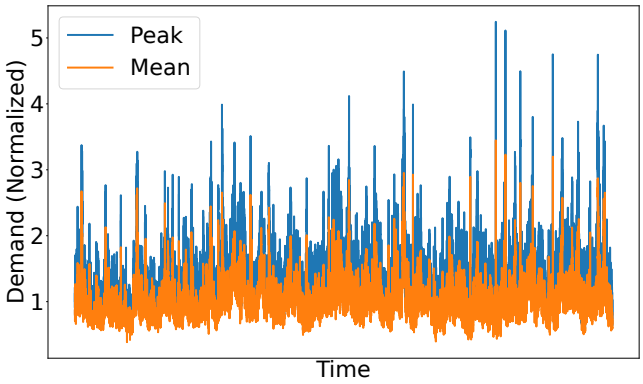


Figure 1: The peak and mean demand of the most active pair in PrivWAN during each one minute period.

2 Background and Motivation

2.1 Why current TE falls short?

Given a network topology, a predicted future matrix, and a set of precomputed tunnels, centralized traffic engineering schemes (e.g., [16,24,26]) determine how traffic must be split across tunnels while optimizing a network-wide objective (e.g., maximizing total carried flow). Conventionally, many of these TE schemes [24,26,34] relied on optimization solvers to determine how to route traffic. More recently, researchers [7, 39, 42] have explored using neural architectures to achieve the same goal. These neural architectures may take a set of past matrices [39,42], or a predicted matrix [6,7] and propose routing for a future matrix. Such neural approaches are more resilient to prediction errors and can lower solving times.

Centralized TE adapts at coarse time-scales. Regardless of these differences, the vast majority of both optimization and neural TE approaches today [7, 16, 24, 26, 34, 42] work with (i) coarse-grained traffic matrices collected over the granularity of tens of seconds or even minutes, and (ii) propose a single split ratio for each source-destination pair, which must perform well until the next recomputation. A central challenge in conducting TE at faster time-scales is not only that they require centralized computation of routes to reflect the new matrix, but also the tables of all routers need to be updated quickly [33,44]. Consequently, these schemes retain the same routing over tens of seconds to a few minutes, indicating the same routing must perform well over all traffic patterns encountered during this period.

Short-term traffic variation is prevalent. To understand short-term fluctuations in TMs, we analyze three weeks of data from PrivWAN, where 5 TMs are collected each minute (although TE is conducted less frequently). We characterize short-term traffic variations by computing the peak and average traffic in each 1-minute period for every source and destination pair. Figure 1 depicts the peak and average at 1-minute granularities for the most active pair (pair with the

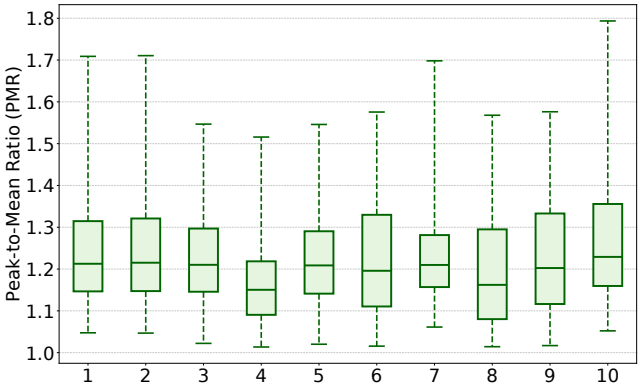


Figure 2: The distribution of Peak to Mean Ratio (PMR) for the ten most active pairs in PrivWAN across minutes. For each pair, PMR is computed every minute. Whiskers indicate the 1st and 99th percentile.

highest traffic volume over the entire period). Figure 2 summarizes the distribution of peak-to-mean ratio (PMR) for the ten most active pairs. The figures show that the peak significantly exceeds the mean – e.g., for the top pair, the median PMR is 1.21 with the 99th%ile being over 1.7. Appendix D further quantifies intra-cycle variation through the Normalized Root Mean Square Error (NRMSE) between the individual TMs and average TM of each cycle. The median NRMSE is 16.1% while the 90th percentile is 28.6%, further substantiating the prevalence of short-term variation.

Coarse-grained TE leads to sub-optimal performance. Unfortunately, owing to traffic variations within a TE cycle, the coarse-grained nature of current TE schemes is sub-optimal. To illustrate, consider Figure 3, which shows an example topology in which two distinct demand matrices (A and B) occur during the same TE cycle due to traffic fluctuations. Figure 3b depicts flow allocation with a centralized scheme that plans for the average of the two matrices. The scheme can route a maximum of 1.5 units for each of the pairs PQ and RQ, and thus only carries a total of 2.5 units for each scenario. We observe that this is the maximum flow that can be sent assuming the split ratios are held constant across scenarios. See Appendix A for a proof.

Observation 1. Any scheme that does not change split ratios across Scenarios A and B can achieve a maximum of 2.5 units on average across the scenarios.

2.2 Illustrating Colt in action

We next discuss Colt's approach for the same example shown in Figure 3. Colt involves both (i) *centrally coordinated* decisions made by a controller at current TE timescales (tens of seconds); and (ii) *local decisions* made by each router at finer timescales as we discuss below.

Centrally coordinated decisions. In each TE cycle, rather than compute a single set of split ratios for each source and destination pair, Colt's central controller computes for each pair (i) a demand threshold; and (ii) two sets of split ratios. Let T_{PQ}, $\mathbf{w^n_{PQ}}$ and $\mathbf{w^b_{PQ}}$ denote the corresponding quantities for pair PQ (the bold notation for $\mathbf{w}$ indicates a vector corresponding to the set of split ratios).

Function 1 Router Local Program

Input: $\mathbf{w^n_{PQ}}$, $\mathbf{w^b_{PQ}}$, T_{PQ}
if $d_{PQ} \leq T_{PQ}$ **then**
  $\mathbf{F_{PQ}} = \mathbf{w^n_{PQ}} * d_{PQ}$
else
  $\mathbf{F_{PQ}} = \mathbf{w^n_{PQ}} * T_{PQ} + \mathbf{w^b_{PQ}} * (d_{PQ} - T_{PQ})$
end if
return $\mathbf{F_{PQ}}$

Local decisions. During the TE cycle, each router uses the above parameters and locally observed information to adapt its routing. Function 1 illustrates the algorithm executed at router P for destination Q. A similar algorithm runs at every ingress router for each destination.

Intuitively, demand d_{PQ} may be viewed as consisting of (i) up to T_{PQ} units of base traffic; and (ii) additional traffic that corresponds to the amount that exceeds T_{PQ}. The base traffic is routed using split ratios $\mathbf{w^n_{PQ}}$, while excess traffic is routed using split ratios $\mathbf{w^b_{PQ}}$. Router P continuously estimates d_{PQ} locally, and uses the algorithm above to compute the net carried flow on each tunnel (denoted by $\mathbf{F_{PQ}}$).

Table 1 presents the optimal values of the T and $\mathbf{w}$ parameters computed by Colt for the example in Figure 3. For instance, for PQ, threshold T is set to 1, all base traffic is routed on the single hop tunnel, and all excess traffic is routed on the PSQ tunnel that contains the shared S-Q link. Figure 4 depicts how the d_{PQ} traffic is split along the two tunnels with these parameter settings.

Demand	$\mathbf{w^n}$	$\mathbf{w^b}$	T
PQ	(100%, 0%)	(0%, 100%)	1
RQ	(100%, 0%)	(0%, 100%)	1

Table 1: Optimal values of thresholds and split ratios as computed by Colt for the example in Figure 3.

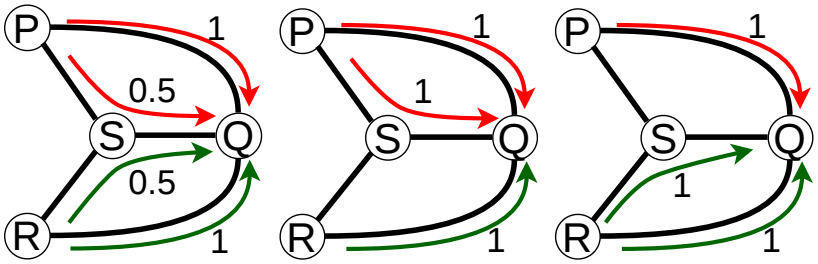
Figure 3c and Figure 3d respectively show how Colt routes traffic for each of Scenarios A and B. Clearly, Colt carries all three units of traffic in both scenarios owing to its ability to more flexibly route traffic based on local information. In contrast, current TE approaches can only route 2.5 units of traffic since they use the same split ratio for all scenarios.

Insights motivating Colt. Colt is designed for typical WAN environments that are generally well provisioned but have some slack capacity to handle traffic bursts.

Colt leverages two observations. First, although individual source and destination pairs exhibit variations at short time-

Scen.	d_{PQ}	d_{RQ}
A	2	1
B	1	2
Avg	1.5	1.5

(a) Demand Scenarios



(b) Centralized TE (avg) (c) Colt (Scenario A) (d) Colt (Scenario B)

Figure 3: Colt vs centralized TE under demand volatility. (a) Multiple demand scenarios A and B occur in the same TE cycle (b) Coarse grained TE uses the same routing for both scenarios which is sub-optimal for each scenario. For example, routing for the average matrix results in only 2.5 units of flow in each scenario. (c)-(d) Colt optimally routes all 3 units of traffic in both scenarios through its local adaptation policies.

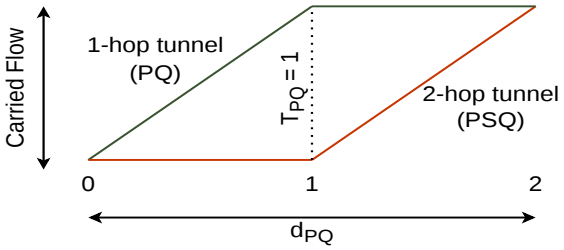


Figure 4: Piecewise linear policy used by Colt to carry traffic from P to Q on the two associated tunnels. The first unit of traffic is routed entirely on tunnel 1, and excess traffic on tunnel 2.

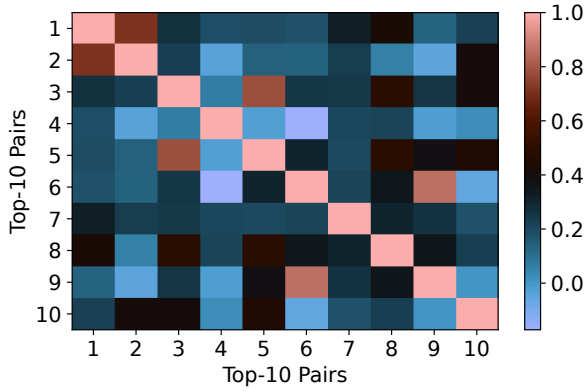


Figure 5: Heatmap showing the Spearman rank correlation in the traffic volumes of the most active source and destination pairs over an example ten-minute period in PrivWAN. Most pairs are weakly correlated.

scales, the correlation across pairs is relatively weak in many cases. To see this, consider Figure 5 which presents the correlation across the ten most active pairs over a representative ten-minute period – the correlations for most pairs is relatively weak. Second, in many cases, not all tunnels of a pair share links with tunnels of another. Colt leverages this by seeking to route normal traffic of each pair on non-shared tunnels to the extent possible, leaving shared paths for more bursty traffic. In Appendix C, we show that the correlation across the top ten pairs is weak over the entire dataset (Figure 20). Further, we characterize the dataset with respect to the number of simultaneously bursty pairs expanding to the top 100 pairs. Our results show the median TE cycle has 5 co-bursty pairs with the 90th percentile across cycles having 10.2 co-bursty pairs. Finally, §5.5 shows that Colt's locally adaptive approach only suffers mild degradation with co-bursts.

3 Colt Design

In this section, we present Colt, a novel framework to handle traffic bursts in WAN TE by allowing each router to use a piecewise linear policy to locally adjust how traffic is routed to each destination. Colt involves new logic in the central controller, as well as each of the ingress routers as shown in Figure 6. Colt comprises three components:

New TE optimization model for richer policies (§3.1). Rather than compute a single set of split ratios in each TE cycle (tens of seconds to multiple minutes), the Colt controller generates parameters of piecewise linear policies deployed in each router. Specifically, in each cycle, the central controller (i) receives H demand matrices $(\mathbf{d}^1, \mathbf{d}^2 \ldots \mathbf{d}^H)$ along with a set of precomputed paths and link capacities; and (ii) generates two sets of split ratios $(\mathbf{w^n_{st}}, \mathbf{w^b_{st}})$ and a threshold (T_{st}) for each source and destination pair, $\langle s,t \rangle$. These parameters are subsequently sent to the corresponding ingress routers, enabling them to dynamically react to traffic bursts using local adaptation.

Novel neural architecture to generate policy parameters (§3.2). A key requirement for Colt is that the generated policy parameters must work well on future (unseen) matrices. Given this, we focus on a neural approach rather than a clas-

sical optimization approach, given recent works [6, 7, 39, 42], that shows the benefits of neural TE schemes in coping with inaccurate predictions. Existing neural architectures, while a starting point, do not address the rich policies that Colt targets. First, while traditional TE involves solving a linear program, Colt's optimization problem is more complex, involving non-linear terms. Second, most neural TE schemes to date focus on optimizing performance for a single future matrix [6, 7, 39, 42]. In contrast, Colt seeks to optimize performance for multiple possible future matrices that may occur in the next TE cycle. We detail Colt's neural architecture in §3.2.

Local adaptation mechanism (§3.3). Using the parameters generated by the Colt controller, ingress routers locally monitor the traffic to each destination, and adjust split ratios based on the traffic to the destination even during the TE cycle. This allows them to dynamically tune split ratios in real-time (§3.3).

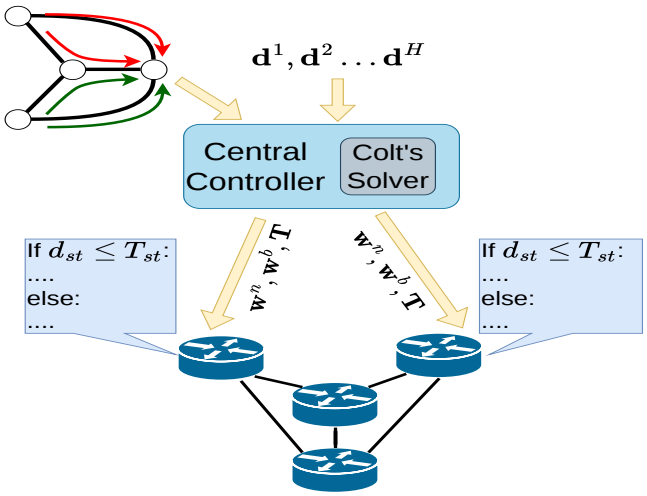


Figure 6: Overview of Colt.

3.1 Formulating Colt's TE problem

We begin by formulating the optimization problem that Colt uses to compute parameters of each router's piecewise linear adaptation policy in a globally coordinated manner. Let $(\mathbf{d}^1, \mathbf{d}^2 \ldots \mathbf{d}^H)$ represent the set of predicted matrices that Colt designs for. By default, we use the matrices in the previous TE cycle as the predicted matrices – however, in Appendix D, we also consider predicted matrices based on linear regression. Colt seeks to determine policy parameters so as to maximize the average carried flow across these matrices as we discuss below. Equation (1) presents the model (see Table 2 for notation).

$$\underset{w^n_{stk}, w^b_{stk}, T_{st}}{\text{maximize}} \quad \text{Average Carried Flow Over } \{d^1_{st}, d^2_{st} \ldots d^H_{st}\} \tag{1a}$$

$$\text{subject to} \quad \sum_k w^n_{stk} \leq 1, \quad \forall \langle s,t \rangle \tag{1b}$$

$$\sum_k w^b_{stk} \leq 1, \quad \forall \langle s,t \rangle \tag{1c}$$

$$\text{Link capacity constraints} \tag{1d}$$

$$d^{h,n}_{st} = \min(d^h_{st}, T_{st}), \qquad \forall \langle s,t \rangle, \forall h \tag{1e}$$

$$d^{h,b}_{st} = \max(0, d^h_{st} - T_{st}), \quad \forall \langle s,t \rangle, \forall h \tag{1f}$$

Partitioning traffic. Equation (1e) and Equation (1f) capture that every demand d^h_{st} (from s to t) may be viewed as consisting of two components: i) normal traffic up to a threshold T_{st}; and ii) traffic that exceeds this threshold. We denote these components by $d^{h,n}_{st}$ and $d^{h,b}_{st}$ respectively. Note that Equation (1e) and Equation (1f) conserve the original value of d^h_{st}. With Colt, routers use a different set of split ratios to route normal and bursty traffic. In particular, for traffic from s to t, let w^n_{stk} and w^b_{stk} respectively denote the fraction of normal and bursty traffic sent on tunnel k. Equation (1b) and Equation (1c) respectively capture that the sum of the w^n and w^b split ratios across all tunnels of a pair is at most 1. Note that not all flow may be carried owing to capacity constraints which we discuss next.

Capacity constraints and objective. To ensure link capacity constraints are honored, we use the equation below:

$$\sum_{s,t,k} \left(w^n_{stk} d^{h,n}_{st} + w^b_{stk} d^{h,b}_{st} \right) P_{ijstk} \leq C_{ij}, \quad \forall \langle i,j \rangle, \forall h \tag{2}$$

This ensures that for every link <i,j>, the aggregate volume of base and excess traffic carried by all tunnels <s,t,k> that traverse the link does not exceed the capacity C_{ij} of the link for any of the traffic matrices d^h.

The objective is to maximize the average carried flow of the set of input demands $\{d^1_{st}, d^2_{st}, \ldots, d^H_{st}\}$. Formally:

$$\max \quad \sum_{h \in H} \sum_{s,t,k} \left(w^n_{stk} d^{h,n}_{st} + w^b_{stk} d^{h,b}_{st} \right) \tag{3}$$

Problem complexity. Unlike classical TE which involves solving linear programs, the formulation above is non-convex since both the objective (Equation (3)) and the link capacity constraints (Equation (2)) have bilinear terms involving products of variables ($w^n_{stk} d^{h,n}_{st}$ and $w^b_{stk} d^{h,b}_{st}$). In addition, while we defer the full mathematical formulation to the Appendix B, demand partitioning requires binary variables to correctly reflect Equation (1e) and Equation (1f). We discuss how Colt tackles the problem complexity next.

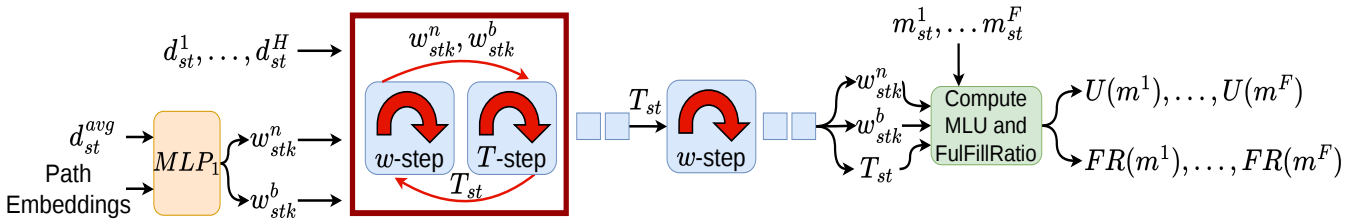


Figure 7: High-level overview of Colt's neural architecture.

Symbol	Description
$\mathcal{D}$	Set of predicted traffic matrices
$\mathcal{M}$	Set of future traffic matrices
d^h_{st}	demand from s to t for predicted matrix $d^h \in \mathcal{D}$
d^{avg}_{st}	average demand from s to t of all $d^h \in \mathcal{D}$
m^f_{st}	demand from s to t for future matrix $m^f \in \mathcal{M}$
$U(d^h), U(m^f)$	Maximum Link Utilization (MLU) for d^h, m^f
$U_{lstk}(d^h)$	Utilization of bottleneck link on stk path for d^h
$FR(m^f)$	Fulfillment Ratio of matrix m^f
T_{st}	Learnable threshold (breaking point) for pair $\langle s,t \rangle$
w^n_{stk}	Split ratio for normal st demand on its k-th path
w^b_{stk}	Split ratio for bursty st demand on its k-th path
g^h_{st}	Binary gate indicator for scenario d^h (1 if $d^h_{st} > T_{st}$, else 0)
P_{ijstk}	Incidence matrix indicating if path <s,t,k> traverses link <i,j>

Table 2: Notation Table

3.2 Neural Architecture for Colt policies

We next motivate why Colt uses a neural approach to tackling the above problem, and present details.

Rationale for a neural approach. The above formulation produces optimal Colt parameters under perfect prediction (i.e., when the matrices that Colt designs for exactly matches the ones encountered in the next TE cycle). Further, under perfect prediction, the capacity constraints in §3.1 guarantee that Colt never exceeds capacity constraints. However, given that predictions are imperfect, we are motivated to explore a neural architecture. The neural approach seeks to optimize traffic successfully delivered on the ground truth matrix, though only provided predicted matrices.

Challenges. In adopting a neural approach, Colt is motivated by the success of prior neural schemes [6, 7, 39, 42, 55] for classical TE approaches. However, while these schemes have tackled linear programs and optimized for a single future matrix, the key challenges for Colt are (i) tackling the complexity associated with non-linear terms; and (ii) optimizing for multiple future TMs in the next TE cycle.

Alternating Optimization Framework. To circumvent the non-linearity introduced by the product of decision variables, Colt employs an alternating optimization framework. The approach isolates the variables, iteratively fixing one set to optimize the other as follows:

1. **Initialization:** We initialize the threshold T_{st} of each source-destination pair to the average volume of that demand across the H predicted traffic matrices (i.e., $T_{st} = d^{avg}_{st}$).

2. **Routing Optimization (w-step):** With T_{st} fixed, the partitioned base and burst demands ($d^{h,n}_{st}$ and $d^{h,b}_{st}$) become constants for all predicted scenarios, and the formulation reduces to a tractable Linear Program (LP). We solve this LP to compute the optimal split ratios w^n_{stk} and w^b_{stk}.

3. **Threshold Optimization (T-step):** Next, we freeze the split ratios w^n_{stk} and w^b_{stk} at their newly computed values. The formulation now reduces to a Mixed-Integer Linear Program (MILP) to find the optimal thresholds T_{st}.

4. **Iteration:** We alternate between the w-step and T-step for K iterations to incrementally refine the policy parameters.

5. **Finalization:** The heuristic concludes with a final w-step. This is necessary to allow Colt to fall back into a traditional, non-adaptive routing if it is optimal. Provably, if $w^n_{stk} = w^b_{stk}$, then T_{st} is effectively ignored, and Colt acts as a standard, non-adaptive scheme. That said, non-adaptive routing is just a special case of Colt.

Aligning neural architecture to alternating framework. Figure 7 shows a high-level overview of the neural architecture of Colt which directly aligns with the alternating framework discussed previously.

The neural architecture consists of two sets of *inner* recurrent units, and an *outer recurrent* unit. The inner units correspond to each of the w-step and T-step of a single round, while the outer unit corresponds to multiple rounds of these steps. The architecture concludes with a w-step as well.

Initialization step. The threshold T_{st} of each st pair is initialized to the average traffic of that pair across the demand matrices $\mathcal{D}$. Further, an MLP is used to generate initial values for each of the w^n_{stk} and w^b_{stk} variables

Demand Splitting and T_{st} Optimization. A critical challenge in training Colt is the traffic splitting logic and T_{st} optimiza-

tion. Naively, this can be done following Equation (4)

$$d_{st}^{h,b} = ReLU(d_{st}^{h} - T_{st})^2 \tag{4a}$$

$$d_{st}^{h,n} = d_{st}^{h} - d_{st}^{h,b} \tag{4b}$$

However, applying ReLU in this case causes the dead gradient problem. Specifically, whenever the neural network predicts a threshold T_{st} that is higher than d_{st}^{h}, the burst component, $d_{st}^{h,b}$, evaluates to zero, and the normal component, $d_{st}^{h,n}$, evaluates to d_{st}^{h}. As a result, the gradient with respect to T_{st} becomes zero, preventing effective learning of T_{st}.

Alternatively, Equation (4) can be rewritten as follows:

$$d_{st}^{h,b} = g_{st}^{h} * (d_{st}^{h} - T_{st}) \tag{5a}$$

$$d_{st}^{h,n} = (1 - g_{st}^{h}) * d_{st}^{h} + g_{st}^{h} * T_{st} \tag{5b}$$

where g_{st}^{h} is a binary variable equal to one if $d_{st}^{h} \geq T_{st}$, and zero otherwise. To circumvent the non-differentiability of our *if* $(d_{st}^{h} \geq T_{st})$ condition, we employ a Straight-Through Estimator (STE) [10], which strictly enforces the binary value of g_{st}^{h} during the forward pass. However, STE generates gradients during the backward pass using a differentiable soft-gate approximation: $g_{st}^{h} \approx \text{sigmoid}(\alpha(d_{st}^{h} - T_{st}))$, where α is a steepness parameter. We initialize α to a low value (e.g., 0.2) to provide smooth gradients during the early stages of training. As training proceeds, we increase α every epoch. This gradually makes the smooth sigmoid steeper, ultimately converging to a step function.

***w*-step.** The w-step takes the set of predicted demand matrices $\mathcal{D}$ and an initial Colt policy (which includes $\mathbf{w^n}$, $\mathbf{w^b}$ and T parameters for each pair), and evaluates the outcome of this policy on each of the matrices $d^h \in \mathcal{D}$. Specifically, for each matrix d^h, and pair st, this involves (i) splitting d_{st}^{h} into a normal and bursty components ($d_{st}^{h,n}$ and $d_{st}^{h,b}$) based on the current T_{st}; (ii) computing utilizations of all links using the split ratios. Next, a *tunnel feature vector (TFV)* is constructed for every matrix d^h, and each tunnel. The TFV of a scenario includes i) the network-wide maximum link utilization ($U(d^h)$), ii) the utilization of the bottleneck link of the tunnel ($U_{lstk}(d^h)$) iii) $d_{st}^{h,n}$ iv) $d_{st}^{h,b}$ v) w_{stk}^{n} vi) w_{stk}^{b}, and vii) suitable path embeddings. Intuitively, a TFV captures how critical a tunnel is for a particular matrix, with high utilizations indicating a parameter change may be needed to accommodate the tunnel under that matrix. To optimize for multiple matrices simultaneously, Colt uses a *set transformer* which takes as input the set of TFVs for each tunnel across scenarios, and creates a *Cross-Scenario Context (CSC)* for that tunnel. Next, an MLP (MLP_w) analyzes the CSC and proposes changes to the w_{stk}^{n} and w_{stk}^{b} parameters for the tunnel stk. Figure 8 shows a schematic diagram of w-step given two scenarios (1 and 2) for simplicity.

***T*-step.** The T-step adjusts the threshold parameter T while keeping split ratios $\mathbf{w^n}$ and $\mathbf{w^b}$ fixed. Following the same procedure above, revised TFVs are computed reflecting changes made in the w-step. A similar set transformer across all scenarios for each tunnel for every pair st produces a multi-TM context. While the $\mathbf{w^n}$ and $\mathbf{w^b}$ parameters are on a per tunnel basis, the T parameter is on a per pair basis requiring a single value shared across k tunnels of a pair st. To capture this, the T-step includes a second set transformer, which takes the cross-scenario contexts across tunnels of each pair, and creates a cross-scenario context for the pair as a whole. This is then fed into an MLP to create a revised T value.

[2] $ReLU(x) = \max(0, x)$

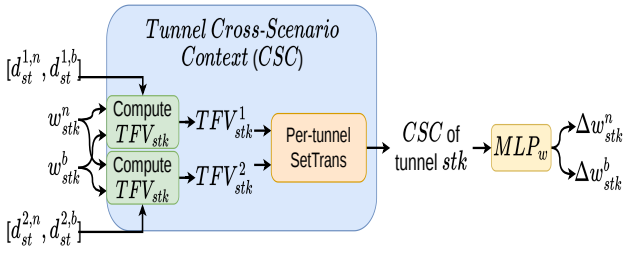


Figure 8: Schematic diagram of the recurrent unit of w-step. For simplicity, two scenarios (1, 2) are used.

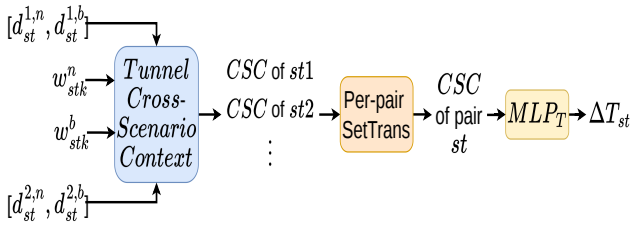


Figure 9: Schematic diagram of the recurrent unit of T-step. For simplicity, two scenarios (1, 2) are used.

Final *w*-step. Following the final T-step made in the recurrent unit, a final w-step is made to adjust w_{stk}^{n} and w_{stk}^{b} given final T_{st}. This is done to align with the heuristic described above and to let Colt be capable of falling back into a traditional non-adaptive routing (by setting $w_{stk}^{n} = w_{stk}^{b}$) if necessary.

Loss function design. Given the generated split ratios and T parameters, we compute one (or more) TE metrics for each matrix m^f in $\mathcal{M}$ (representing the ground truth matrices in the next TE cycle) assuming appropriate per-router local adaptation consistent with the Colt parameters. The loss reflects the average of these metrics across the matrices. This is in contrast to existing neural schemes that only compute loss on a single matrix for the entire cycle. We focus on the FulfillRatio metric [6], which captures the total traffic delivered for the ground truth matrix after taking into account any capacity violations and simulating the effect of router packet drop policies. The metric is computed using a flow-level simulator from prior work [6] (summarized in §4). Since the FulfillRa-

tio metric is not effective in well-provisioned networks (since many different routings can result in all traffic being carried), we complement it with the MLU metric. Specifically, the loss function prioritizes MLU and uses FulfillRatio as secondary using the gradient projection method from [6].

Implementation details. As shown in Figure 7, Colt leverages path embeddings as inputs, building upon prior work [7]. This involves two key neural modules: (i) a graph neural network that feeds the network topology along with capacities, generating a rich embedding of edges; and (ii) a set transformer that utilizes these edge embeddings to yield embeddings of tunnels. We implemented Colt using PyTorch [41] and used Adam [31] optimizer to learn its weights.

3.3 Local Adaptivity

We have discussed so far how Colt computes its policy parameters (w^n_{stk}, w^b_{stk}, and T_{st}) at coarse time-scales (minute or more). Each router uses these parameters to adapt to traffic bursts in real time during the TE cycle. To achieve this, each router (i) tags traffic associated with each s and t pair as normal or bursty based on whether the T_{st} threshold is exceeded; and (ii) applies the split ratio $\mathbf{w^n}$ to the normal traffic, and $\mathbf{w^b}$ to bursty traffic. Tagging can be achieved using a traffic meter for each pair [22, 23, 40, 46] (typically based on a token bucket scheme), a common approach for differential treatment of traffic based on sending rates. The forwarding table of an ingress router stores split ratios for each pair and tag combination (similar to per-class per-pair split ratios of multi-class TE schemes [24, 26]).

4 Evaluation Methodology

Datasets. We primarily conducted experiments with PrivWAN, a real-world private WAN with several tens of nodes and several hundreds of edges. The dataset spans three weeks, and logs five matrices per minute. Further, it includes the topology (nodes, edges, and link capacities) at one-minute granularity. Note that capacity variations occur throughout the dataset owing to partial and full link unavailability (owing to failures or planned maintenance), and organic growth. We also conduct experiments with GEANT [1, 47] for which traffic datasets are publically available.

Schemes compared. We compare Colt with several schemes that we discuss next.

Optimization-based schemes. NonAdap-Opt [58] is an optimization-based scheme that given a set of matrices computes a set of split ratios (fraction of traffic sent on each tunnel for each pair) that maximize the average flow carried for those matrices while ensuring capacity constraints are met for all the matrices. Note that (i) using the same split ratio for all matrices is representative of typical centralized TE schemes; and (ii) among schemes that use the same split ratios during a TE cycle, NonAdap-Opt is optimal if given the ground truth (the set of matrices encountered in each TE cycle).

Neural schemes. We compare Colt with three representative and recent neural TE schemes: DOTE [42], FIGRET [39] and Hattrick [6] which have been shown to work well in the presence of prediction error. All these schemes are centralized, producing a fixed split ratio for an entire TE cycle. Given past input matrices, DOTE uses a simple DNN to determine the split ratios for a future matrix. FIGRET [39] uses the same neural architecture as DOTE [42], but seeks to address traffic variation by adjusting the loss function to encourage traffic spread across multiple tunnels for historically bursty flows. We consider Hattrick because the PrivWAN dataset involves topology changes, and Hattrick is a state-of-the-art scheme that can handle such changes. While Hattrick can handle multiple traffic classes, we focus our evaluations on a single traffic class. In addition, we compare with RedTE [21], a distributed neural TE scheme. RedTE models TE as a cooperative multi-agent reinforcement learning (RL) system (with each router being an agent). Each router runs a neural network based on its local information during the inference phase. The training phase involves a global critic network that takes as input the entire network topology and traffic matrix, as well as the local routing configurations generated by each router's neural network.

Methodology and metrics. In each TE cycle, Colt and most baseline schemes take the past five traffic matrices as input (TMs in the previous cycle) and generates decisions for the next TE cycle. Our default experiments for PrivWAN use a 1-minute TE-cycle with 5 matrices per cycle. Appendix F simulates different TE-cycle lengths, highlighting the impact of intra-cycle traffic variation on performance.

For DOTE, FIGRET and Hattrick, the decisions are the fixed split ratio used for all of the next TE cycle. To achieve this, the schemes are trained with the loss functions of each scheme, but applied over the average TM in the next TE cycle. The inference stage produces split ratios given the TMs in the past cycle, and this split ratio is used for each TM in the next cycle. Since Hattrick is only designed to take a single matrix as input, we feed it the average of the past five matrices.

In contrast, Colt generates the parameters (w^n_{stk}, w^b_{stk}, and T_{st}) of each pair's piecewise linear policy for the next TE cycle. Each router determines which split ratio to apply for each pair during the cycle based on whether traffic is within the T_{st} threshold as tagged by a traffic meter (§3.3). Since packet level traces are unavailable to us, our evaluations assume that w^n_{stk} is applied on upto T_{st} units of traffic for each TM in the cycle, and w^b_{stk} on any excess.

RedTE is a distributed TE scheme that leverages per-router neural models operating on locally available information to generate real-time decisions. Given the most recent traffic matrix (potentially within the same TE cycle), RedTE computes split ratios to be applied to the subsequent traffic matrix.

All our experiments use the K shortest paths as tunnels

for all node pairs. We use K=13 for PrivWAN, and K=4 for GEANT. Among the schemes we compare, optimization-based schemes (NonAdap-Opt and Colt-Opt) are overly conservative since they determine split ratios feasible for each of the TMs in the past cycle. In contrast, many neural approaches optimize for MLU and are overly aggressive disregarding capacity constraints during optimization. To ensure fairness of comparisons, we rescale the split ratios of all schemes to be feasible for the average TM in the past cycle. Further, we focus on the FulfillRatio metric introduced in [6] which captures the effective flow successfully delivered for each pair after adjusting for link capacity violations.

We compute FulfillRatio using a flow-level simulator used in [6] which takes as input the per-tunnel split ratios produced by a scheme. The simulator (i) determines all tunnels that traverse a link whose capacity is exceeded for the ground truth matrix; and (ii) scales down the allocation to each such tunnel by the utilization of its most congested link. Finally, we normalize all results with respect to the FulfillRatio that the (oracle) optimal scheme would obtain for a ground truth matrix if designed solely for that matrix. We refer to this normalized value as NormFulFill, which can achieve a maximum value of 1.

Colt could lead to out-of-order delivery since split ratios may vary during a TE cycle. Such out-of-order delivery may also occur in non-adaptive TE when adjusting split ratios across TE cycles, but may potentially occur more frequently with Colt. In practice, techniques such as flowlet switching can mitigate the impact [30, 50]. Further, there is growing interest in packet spraying techniques with hardware support for resequencing [2, 12, 17, 45, 48]. We defer an investigation of these issues to future work.

5 Results

5.1 Colt vs. alternatives

We begin by comparing Colt against existing optimization-based and neural TE schemes PrivWAN. Since some of the baselines (DOTE and FIGRET) are not designed for datasets where link capacities vary [7], we evaluate these schemes on a variant of PrivWAN (which we refer to as PrivWAN-FixedTopo). This variant is the same topology as PrivWAN, but link capacities are kept the same throughout the trace. We achieved this by using the same topology snapshot for all TMs.

Figure 10 shows that on PrivWAN, Colt significantly outperforms both NonAdap-Opt (the state-of-the-art optimization approach) and Hattrick. For example, Colt achieves a median NormFulFill of 0.994, while Hattrick and NonAdap-Opt only achieve 0.964 and 0.977 respectively. This corresponds to improvements of 3.0 and 1.7 percentage points for Colt over Hattrick and NonAdap-Opt, respectively, and bridges

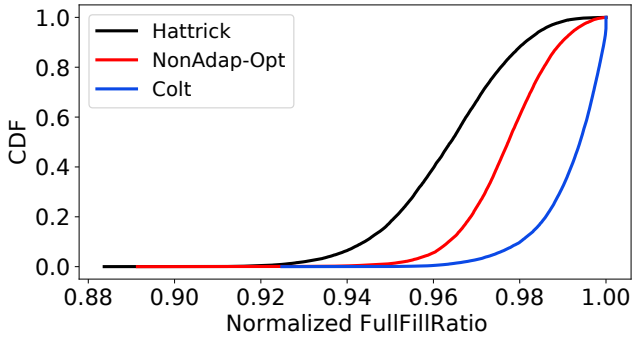


Figure 10: Comparing Colt to alternatives on PrivWAN.

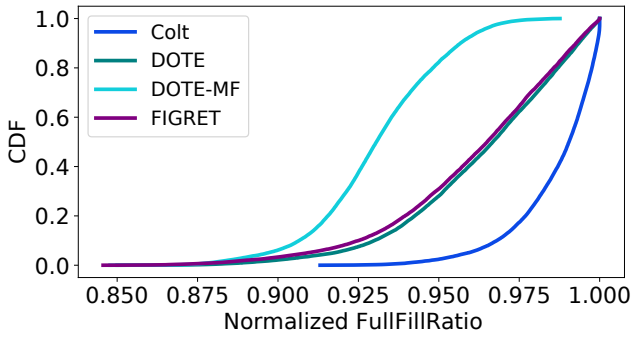


Figure 11: Comparing Colt to alternatives on PrivWAN-FixedTopo.

83% of the gap between Hattrick and an oracle that optimally routes traffic with exact knowledge of the TM (NormFulFill = 1), and 74% of the gap between NonAdap-Opt and the oracle.

Colt outperforms for a few reasons. First, unlike both baselines, which fix the split ratios throughout a TE cycle, Colt generates locally adaptive policies and can adjust the split ratios during the cycle. Second, relative to NonAdap-Opt, Colt's neural approach can better handle imprecise predictions, and optimize the FulfillRatio metric after accounting for congestion effects. Third, relative to Hattrick which is only designed for a single TM, Colt's neural architecture is explicitly designed to optimize multiple potential future TMs.

Surprisingly, Hattrick performs worse than NonAdap-Opt, indicating that the benefits of the multi-TM optimization with NonAdap-Opt outweigh the other benefits of Hattrick's neural approach. Colt, in contrast, combines both these, which explains its improved performance over both baselines.

Figure 11 compares Colt with DOTE and FIGRET on PrivWAN-FixedTopo. Since DOTE supports both the MaxFlow and MLU metrics, we present results with both DOTE variants (that we refer to as DOTE-MF and DOTE respectively). Colt outperforms all schemes achieving a median NormFulFill of 0.991, while the corresponding values for DOTE-MF, DOTE, and FIGRET are 0.931, 0.967, and 0.965 respectively. Further, Colt is always above 0.91, while the other systems see NormFulFill lower than 0.86 in some

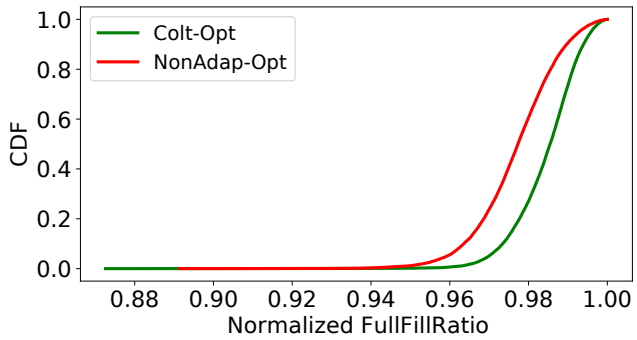


Figure 12: Benefits of local adaptivity. Comparing Colt-Opt, a system variant to NonAdap-Opt on PrivWAN.

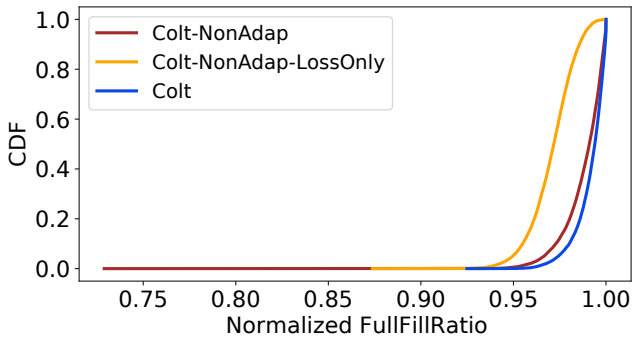


Figure 13: Ablation study comparing Colt with its variants; Colt-NonAdap and Colt-NonAdap-LossOnly.

cases.

The reasons for the out-performance is again because both DOTE variants and FIGRET use fixed split ratios during a TE cycle, and are not explicitly designed to optimize for multiple future TMs. DOTE-MF does not perform as well as DOTE because, while the scheme generates rate caps that are guaranteed to meet link capacity constraints on future matrices, it is too conservative if the predicted demands were lower than actual [6]. DOTE and FIGRET optimize the MLU metric (which does not take capacity constraints into account), but we note that the NormFulFill values correspond to final delivered traffic with the schemes reported by the flow-level simulator (§4) which models capacity constraints.

While not shown, we have confirmed Colt-Opt and Hattrick do not perform as well as Colt on PrivWAN-FixedTopo as well. Interestingly, Hattrick performs a bit worse than DOTE and FIGRET even though it explicitly optimizes the Fulfill-Ratio. A potential reason is that Hattrick takes a single matrix as input (the average matrix of the previous cycle), while DOTE and FIGRET take multiple input matrices though only optimize for one future matrix.

5.2 Understanding Colt's benefits

There are two key benefits of Colt: (i) its local adaptivity approach instead of constant split ratios in a TE cycle; and (ii) a neural architecture explicitly optimized for multiple TMs. We evaluate each of these in turn.

Benefits of local adaptivity. We introduce Colt-Opt, a variant of Colt which also uses locally adaptive policies, but does not have the benefits of a neural approach. Specifically, the scheme uses the optimization model in §3.1, and solves it directly using an optimization solver and an alternating optimization method instead of a neural network. Figure 12 compares NonAdap-Opt and Colt-Opt. The figure shows that Colt-Opt outperforms NonAdap-Opt, demonstrating the benefits of a locally adaptive policy. The median NormFulFill for Colt-Opt and NonAdap-Opt is 0.985 and 0.977, which reduces the gap between NonAdap-Opt and ideal by 35%. Further, Appendix D shows that NonAdap-Opt's performance is not improved with a linear regression predictor instead of the previous cycle's TMs. Finally Appendix F shows Colt-Opt outperforms even under perfect prediction indicating the inherent benefits of local adaptivity.

Benefits of explicitly optimizing for multiple TMs in the neural architecture. To explore the benefits of Colt's architecture to optimize for multi TMs in a cycle, we consider two Colt variants: Colt-NonAdap and Colt-NonAdap-LossOnly. Colt-NonAdap uses the same multi-TM architecture and loss function as Colt, except that it is restricted to fixed split ratios. Colt-NonAdap-LossOnly further simplifies Colt-NonAdap by not having architectural constructs to handle multiple TMs, although the loss function is identical to Colt. It takes a single TM as input (the average TM in the past cycle). Figure 13 shows that Colt-NonAdap-LossOnly performs much worse, indicating the importance of an architecture capable of explicit optimization over a set of matrices. Further, Colt performs better than Colt-NonAdap especially at the tail, owing to the locally adaptive policies that it generates.

5.3 Colt vs RedTE

We start with a qualitative comparison of Colt and RedTE.

Qualitative comparisons. Table 3 qualitatively compares

	Other baselines	RedTE	Colt
Recent TMs for global coordination?	Yes	No	Yes
Adapt routes locally during TE cycle?	No	Yes	Yes
Interpretable local adaptation?	N/A	No	Yes
Topology changes?	Scheme dependent	No	Yes

Table 3: Qualitative comparison between Colt, RedTE, and other centralized TE schemes.

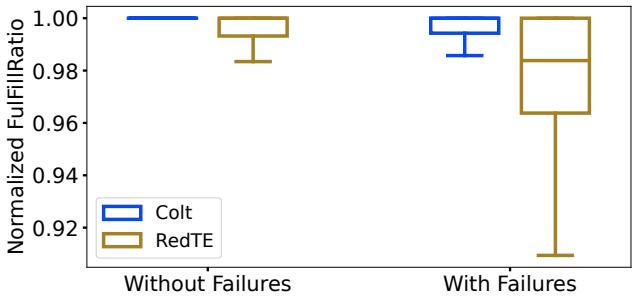


Figure 14: Performance comparison between RedTE and Colt on the Abilene topology.

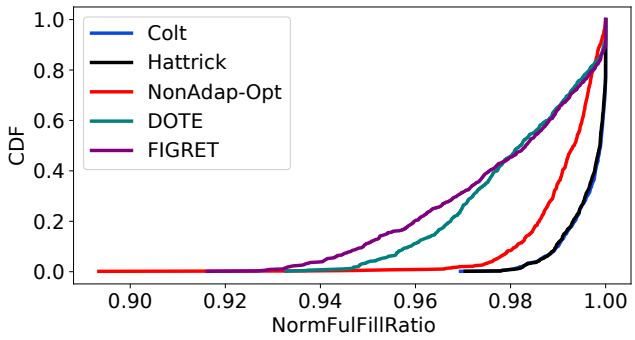


Figure 15: Comparing Colt to other schemes on the GEANT dataset.

Colt and RedTE. Unlike other TE baselines, RedTE does allow routers to adapt during a TE cycle as each router locally runs a neural network. However, RedTE only has access to global information at training time (when a global critic network is trained). Since training is expensive, models are only updated over hours implying they do not incorporate information about more recent network state (link failures, or recent traffic matrices). While inference happens in real time, each router makes decisions only using local information and it is unclear how to incorporate recent global network state such as failure information and TMs into RedTE's local neural models. Further, since routers run a neural network, the actual policy used is not easily interpretable. In contrast, Colt exploits recent global state (topology and TM) available at the start of each TE cycle, with routers running simple and interpretable piecewise linear policies.

Quantitative comparisons. We focus our comparisons with RedTE on the Abilene topology since for PrivWAN and GEANT, the time taken to train models using the publicly available author-provided codebase [3] was prohibitive. This is consistent with recent work [38] pointing to scaling challenges with the RedTE codebase.

Our analysis of the code points to two reasons: CPU-heavy computation (e.g., per-TM loops to aggregate traffic over links), and circular training that examines each TM K times per epoch (100 or 1000 by default). A single epoch for one hyperparameter setting would thus take weeks on PrivWAN (9 seconds per step, 30K TMs) and days on GEANT (0.65 seconds per step, 6K TMs).

We train and test RedTE on the Abilene topology using data provided by the author codebase [3]. We used 4000 TMs for training, 500 for validation and 500 for testing, scaling TMs to ensure the average MLU was about 0.8 (closer to a well utilized but sufficiently provisioned network). Groups of five TMs were considered to form a single TE cycle with Colt. We used the RedTE code to conduct a hyperparameter search, and picked the model with the best validation performance.

Figure 14 (left) presents boxplots that compares RedTE and Colt. The figure shows that Colt achieves optimal NormFulFill in almost all samples, but RedTE falls short for more than 25% of the samples. We believe this is due to RedTE not incorporating recent global network state. Next, we consider how the lack of recent topology state impacts the performance of RedTE under failures. To this end, we take the same RedTE and Colt models trained without failure, and test both the models under failure. Figure 14 (right) shows a distribution of the NormFulFill across all combinations of test set TMs and failure scenarios (omitting failures that disconnected the graph) for both schemes. For RedTE, we use local proportional rescaling [42], which ensures traffic is redirected from tunnels with failed links. In contrast, Colt's neural model incorporates topology state at the start of each TE cycle, and the neural inference produces routing parameters that avoid failed tunnels. The figure shows that Colt achieves an even larger outperformance over RedTE under failure improving the 25th percentile NormFulFill from 0.96 to 0.99.

5.4 Sensitivity to dataset

So far, we have focused our evaluations primarily on PrivWAN because the dataset includes multiple TMs collected per minute. Unfortunately, GEANT has TMs collected at coarse 15 minute granularities. Since each TM is a long term average, the dataset is less likely to include significant traffic bursts. Indeed, the average peak-to-mean ratio (PMR) for the most active pair was only 1.04 in contrast to PrivWAN where it was 1.23. Figure 19 in the Appendix summarizes the PMR of the top 10 pairs.

Figure 15 shows that Colt outperforms most schemes, though Hattrick performs well, and comparably to Colt. We believe two factors potentially played a role in Hattrick performing well. Besides the lack of burstiness in the data, we observed the network was not well provisioned with most MLU values exceeding 1, and 29% of the values exceeding 2. We conducted a targeted experiment where we scaled down TMs so a majority of TMs (about 60%) had an MLU under 1 with a median MLU of about 0.9, more typical of WANs today. Figure 16 shows Colt outperforms Hattrick in this setting – e.g., the 25th percentile NormFulFill is improved from 0.979

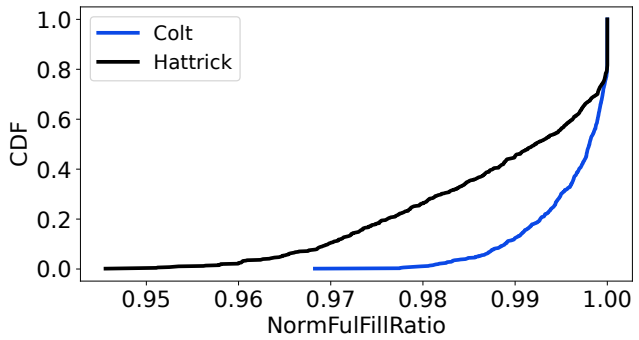


Figure 16: Comparison between Colt and Hattrick on GEANT with the matrices scaled down to ensure the majority of TMs had an MLU under 1.

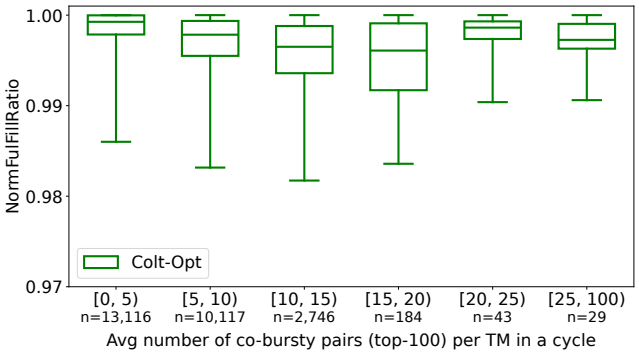


Figure 17: Colt-Opt's NormFulFill grouped by the average number of co-bursty top-100 pairs per TM in each TE cycle, assuming optimal prediction. *n* represents the number of TE cycles per bucket.

to 0.994, reducing the gap with an ideal scheme by 71%.

5.5 Other sensitivity studies

Impact of correlated bursts. In Section 2.2, we indicated that Colt strives to send non-bursty traffic on non-shared paths, and bursty traffic on shared ones. A natural question is how this local mechanism performs when many pairs burst simultaneously, since their excess traffic might then compete for the same shared links. To that end, we characterize bursts in PrivWAN focusing on the top 100 pairs by volume. A pair in a TM is considered bursty if its demand is significantly higher than its average demand over a 10-minute window (see Appendix C for details).

Figure 17 shows the performance of Colt-Opt with TE cycles grouped by the average number of co-bursty pairs per TM in the cycle. We use Colt-Opt under perfect prediction to isolate co-bursts from prediction error and learning. NormFulFill degrades only mildly with more co-bursts. The median falls from 0.999 (first group) to 0.996 (fourth group). Interestingly, Colt-Opt performs slightly better in the TE cycles with the two highest co-burst levels (two rightmost boxplots). Many of these TE cycles have lower traffic volume, so more pairs tend to exceed their mean. Further, there are relatively few TE cycles with these co-burst levels, making definitive conclusions harder.

We separate the effect of co-bursts from that of traffic volume in Appendix C(Figure 22). Co-bursts slightly degrade Colt-Opt's performance at every traffic volume level, and the degradation increases with traffic volume. However, even in the worst case (highest traffic volume and the most co-bursts), the mean NormFulFill remains above 0.99, indicating an acceptable performance drop.

Sensitivity to prediction error. We characterize prediction error in Appendix D – with a linear regression predictor, the median Normalized Root Mean Square Error is 9.3% for the average TM within a cycle, and 18.8% for individual TMs. Note the higher prediction error for individual TMs owing to intra-cycle variability. While this already shows Colt is effective under prediction error, we stress its performance with additional controlled perturbation in Appendix E. Colt is more resilient than NonAdap-Opt and Hattrick. As prediction error grows, the median NormFulFill of Colt only drops from 0.994 to 0.974, compared with 0.964 to 0.911 for Hattrick and 0.977 to 0.926 for NonAdap-Opt.

Sensitivity to TE cycle length. While we have focused on one minute TE cycles, Figure 27 (Appendix F) considers the impact of larger TE cycles on Colt's local adaptivity. To remove confounding effects related to prediction error, we focus on Colt-Opt and perfect prediction. The results show the median of Colt-Opt degrades only slightly (from 0.999 to 0.995) when increasing TE cycle lengths from 1 to 5 minutes, while NonAdap-Opt degrades more (from 0.992 to 0.977). Further, Colt-Opt's wins are more pronounced at the tail.

5.6 Computation Time

We evaluate the computational efficiency of Colt's neural approach in generating traffic engineering policies within each TE cycle. Given the complexity of optimizing network-wide routing decisions, our primary objective is to determine whether Colt can generate high-quality policies within practical time constraints compared to conventional centralized optimization schemes.

Figure 18 presents Colt's runtime across PrivWAN, as well as two larger topologies: UsCarrier (158 nodes, 378 edges), and KDL (754 nodes, 1790 edges) [32]. For the latter two topologies, we employed synthetic traffic matrices generated using the methodology from [42] due to unavailability of real-world data. All Colt and Hattrick experiments were conducted on an NVIDIA RTX PRO 6000 Blackwell GPU with 96GB memory. In contrast, NonAdap-Opt was implemented using Gurobi optimizer on an AMD EPYC 7763 processor (64 cores) with 128GB RAM.

Our experiments reveal three findings: First, Colt demon-

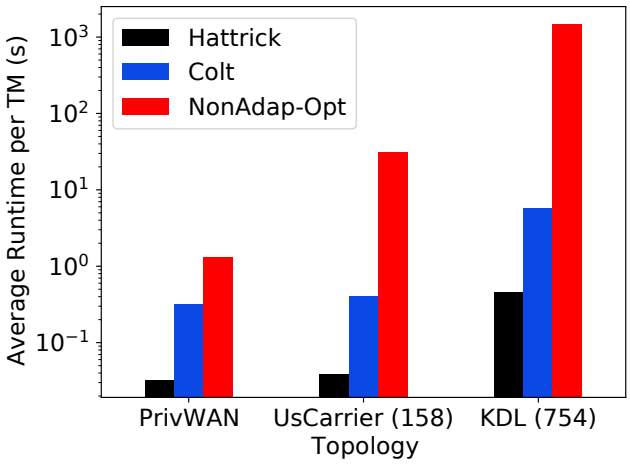


Figure 18: Comparing computation times of schemes across different topologies.

strates practical feasibility, completing policy generation in under 6 seconds even for the KDL topology (our largest network), which aligns well with typical TE cycle durations. Second, Colt achieves two orders of magnitude speedup over NonAdap-Opt, reducing solution time from 1470 seconds to less than 6 seconds on KDL. Third, Colt is moderately slower than Hattrick, a cost justified by its better performance.

Finally, we note that alternative approaches like DOTE and FIGRET exhibit lower computation times but are constrained by their inability to handle dynamic topologies, optimize for future traffic matrices, or support adaptive routing.

6 Related work

dSDN [33] distributes TE optimization across routers to improve fault tolerance and reduces dependency on controller software. However, split ratios are fixed between periodic recomputations which are performed at coarse timescales since TMs must be collected and disseminated to all routers. Colt is orthogonal as it focuses on richer local adaptive mechanisms between recomputations. An interesting future direction is to integrate Colt-style policies within dSDN.

TexCP [29] is a notable fully distributed TE scheme, where ingress routers make local decisions on how traffic must be split based on feedback from routers that monitor link utilizations and queue states. TexCP does not involve any centralized optimization with network-wide views owing to which it can take multiple iterations to converge [21]. In contrast, Colt still uses central TE controllers, except that the policies it deploys at ingress routers allow for more flexibility than fixed routes.

Dealing with traffic uncertainty has received much attention in the community [8, 9, 52, 58]. Given a set of demands (either explicitly enumerated, or an uncertainty set), these works come up with a fixed routing that either optimize the worst case [8, 9, 52] or the average performance [58]. SMORE [34] preselects good quality paths by decomposing an oblivious routing, and recomputes split ratios in each TE cycle. However, the split ratios are unchanged during a cycle. Colt is orthogonal focusing on locally adaptive policies rather than fixed split ratios, and exploring the advantages of a neural approach. We focus on the average performance because the worst-case is overly conservative [11, 13, 27, 34, 54, 58]. However, it is easy to adapt Colt for the worst-case in each TE cycle by adapting the objective and loss function.

More generally, designing for multiple (traffic or failure) scenarios using robust optimization is well studied [14, 15, 18, 19, 28, 36, 43, 53]. TE schemes quickly react to failures by local rescaling at the ingress [28, 36, 53], a form of local adaptation. In contrast, Colt handles WAN traffic bursts, and the parameters of its local policies are centrally coordinated. Researchers have extensively studied how to deal with microbursts in data center networks (e.g., [5]) – in contrast our focus is on tackling the coarse nature of WAN TE

Colt's neural approach builds on a fast emerging class of ML TE algorithms [6, 7, 20, 37–39, 42, 49, 55, 56, 59]. Colt's novelty is in tackling the non-convexity in generating the richer locally adaptive policies, and optimizing for multiple matrices in the next TE cycle.

7 Conclusion

In this paper, we have made three contributions in the context of Colt[3], a new system to quickly handle traffic variations during a WAN TE cycle. (i) a new design point where ingress routers use a piecewise linear policy to dynamically adjust traffic splits during a TE cycle while only leveraging locally available information. (ii) a new optimization model for generating parameters of each router's local policy in a centrally coordinated manner; (iii) a custom neural architecture that tackles the non-convexity of the model using an alternating framework while optimizing for multiple matrices in the next TE cycle. Our evaluations on PrivWAN show that Colt bridges 74% of the gap between NonAdap-Opt (the best optimization-based non-adaptive approach) and an oracle. Further, Colt bridges 83% of the gap between Hattrick and an oracle, while also significantly outperforming recent neural TE approaches, including DOTE, FIGRET, and RedTE.

This work does not raise any ethical issues.

Acknowledgments. We thank our shepherd and the anonymous reviewers for their feedback, which helped to improve the paper. This work was funded in part by the National Science Foundation (NSF) award 2504819 and the Krenicki Center for Business Administration and Machine Learning.

[3] Code available at: https://github.com/Purdue-ISL/Colt

References

[1] GEANT network. http://geant3.archive.geant.net/Network/NetworkTopology/pages/home.aspx.

[2] Powering next-generation AI networking with NVIDIA SuperNICs. https://developer.nvidia.com/blog/powering-next-generation-ai-networking-with-nvidia-supernics/. NVIDIA Technical Blog.

[3] Redte codebase. https://github.com/NASP-THU/RedTE.

[4] Firas Abuzaid, Srikanth Kandula, Behnaz Arzani, Ishai Menache, Matei Zaharia, and Peter Bailis. Contracting wide-area network topologies to solve flow problems quickly. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 175–200. USENIX Association, April 2021.

[5] Mohammad Alizadeh, Tom Edsall, Sarang Dharmapurikar, Ramanan Vaidyanathan, Kevin Chu, Andy Fingerhut, Vinh The Lam, Francis Matus, Rong Pan, Navindra Yadav, and George Varghese. Conga: distributed congestion-aware load balancing for datacenters. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, SIGCOMM '14, page 503–514, New York, NY, USA, 2014. Association for Computing Machinery.

[6] Abd AlRhman AlQiam, Zhuocong Li, Satyajeet Singh Ahuja, Zhaodong Wang, Ying Zhang, Sanjay G. Rao, Bruno Ribeiro, and Mohit Tawarmalani. Hattrick: Solving multi-class te using neural models. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25, page 264–278, New York, NY, USA, 2025. Association for Computing Machinery.

[7] Abd AlRhman AlQiam, Yuanjun Yao, Zhaodong Wang, Satyajeet Singh Ahuja, Ying Zhang, Sanjay G. Rao, Bruno Ribeiro, and Mohit Tawarmalani. Transferable neural wan te for changing topologies. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 86–102, New York, NY, USA, 2024. Association for Computing Machinery.

[8] David Applegate, Lee Breslau, and Edith Cohen. Coping with network failures: Routing strategies for optimal demand oblivious restoration. In *Proceedings of the Joint International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '04/Performance '04, pages 270–281, 2004.

[9] David Applegate and Edith Cohen. Making intra-domain routing robust to changing and uncertain traffic demands: Understanding fundamental tradeoffs. In *Proceedings of ACM SIGCOMM*, pages 313–324, 2003.

[10] Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *CoRR*, abs/1308.3432, 2013.

[11] Jeremy Bogle, Nikhil Bhatia, Manya Ghobadi, Ishai Menache, Nikolaj Bjorner, Asaf Valadarsky, and Michael Schapira. Teavar: Striking the right utilization-availability balance in wan traffic engineering. In *Proceedings of ACM SIGCOMM*, 2019.

[12] Tommaso Bonato, Abdul Kabbani, Ahmad Ghalayini, Michael Papamichael, Mohammad Dohadwala, Lukas Gianinazzi, Mikhail Khalilov, Elias Achermann, Daniele De Sensi, and Torsten Hoefler. Reps: Recycled entropy packet spraying for adaptive load balancing and failure mitigation. In *Proceedings of the 21st European Conference on Computer Systems*, EUROSYS '26, page 225–246, New York, NY, USA, 2026. Association for Computing Machinery.

[13] Yiyang Chang, Chuan Jiang, Ashish Chandra, Sanjay Rao, and Mohit Tawarmalani. Lancet: Better network resilience by designing for pruned failure sets. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 3:1–26, 12 2019.

[14] Yiyang Chang, Sanjay Rao, and Mohit Tawarmalani. Robust validation of network designs under uncertain demands and failures. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 347–362, 2017.

[15] Chandra Chekuri. Routing and network design with robustness to changing or uncertain traffic demands. *SIGACT News*, 38(3):106–129, 2007.

[16] Marek Denis, Yuanjun Yao, Ashley Hatch, Qin Zhang, Chiun Lin Lim, Shuqiang Zhang, Kyle Sugrue, Henry Kwok, Mikel Jimenez Fernandez, Petr Lapukhov, Sandeep Hebbani, Gaya Nagarajan, Omar Baldonado, Lixin Gao, and Ying Zhang. Ebb: Reliable and evolvable express backbone network in meta. In *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM SIGCOMM '23, page 346–359, New York, NY, USA, 2023. Association for Computing Machinery.

[17] Advait Dixit, Pawan Prakash, Y. Charlie Hu, and Ramana Rao Kompella. On the impact of packet spraying in data center networks. In *2013 Proceedings IEEE INFOCOM*, pages 2130–2138, 2013.

[18] N. G. Duffield, Pawan Goyal, Albert Greenberg, Partho Mishra, K. K. Ramakrishnan, and Jacobus E. van der Merive. A flexible model for resource management in virtual private networks. In *Proceedings of ACM SIGCOMM*, pages 95–108, 1999.

[19] Bernard Fortz and Mikkel Thorup. Robust optimization of OSPF/IS-IS weights. In *Proceedings of International Network Optimization Conference*, pages 225–230, 2003.

[20] Nan Geng, Mingwei Xu, Yuan Yang, Chenyi Liu, Jiahai Yang, Qi Li, and Shize Zhang. Distributed and adaptive traffic engineering with deep reinforcement learning. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, pages 1–10, 2021.

[21] Fei Gui, Songtao Wang, Dan Li, Li Chen, Kaihui Gao, Congcong Min, and Yi Wang. Redte: Mitigating subsecond traffic bursts with real-time and distributed traffic engineering. ACM SIGCOMM '24, page 71–85, New York, NY, USA, 2024. Association for Computing Machinery.

[22] J. Heinanen and R. Guerin. A single rate three color marker. RFC 2697, 1999.

[23] J. Heinanen and R. Guerin. A two rate three color marker. RFC 2698, 1999.

[24] Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill, Mohan Nanduri, and Roger Wattenhofer. Achieving high utilization with software-driven wan. In *Proceedings of ACM SIGCOMM*, pages 15–26, 2013.

[25] Chi-Yao Hong, Subhasree Mandal, Mohammad Al-Fares, Min Zhu, Richard Alimi, Kondapa Naidu B., Chandan Bhagat, Sourabh Jain, Jay Kaimal, Shiyu Liang, Kirill Mendelev, Steve Padgett, Faro Rabe, Saikat Ray, Malveeka Tewari, Matt Tierney, Monika Zahn, Jonathan Zolla, Joon Ong, and Amin Vahdat. B4 and after: Managing hierarchy, partitioning, and asymmetry for availability and scale in google's software-defined wan. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 74–87, 2018.

[26] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, Jon Zolla, Urs Hölzle, Stephen Stuart, and Amin Vahdat. B4: Experience with a globally-deployed software defined wan. In *Proceedings of ACM SIGCOMM*, pages 3–14, 2013.

[27] Chuan Jiang, Zixuan Li, Sanjay Rao, and Mohit Tawarmalani. Flexile: Meeting bandwidth objectives almost always. In *Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '22, page 110–125, New York, NY, USA, 2022. Association for Computing Machinery.

[28] Chuan Jiang, Sanjay Rao, and Mohit Tawarmalani. Pcf: Provably resilient flexible routing. In *Proceedings of ACM SIGCOMM*, page 139–153, 2020.

[29] Srikanth Kandula, Dina Katabi, Bruce Davie, and Anna Charny. Walking the tightrope: responsive yet stable traffic engineering. *SIGCOMM Comput. Commun. Rev.*, 35(4):253–264, August 2005.

[30] Srikanth Kandula, Dina Katabi, Shantanu Sinha, and Arthur Berger. Dynamic load balancing without packet reordering. *ACM SIGCOMM Computer Communication Review*, 2007.

[31] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[32] Simon Knight, Hung Nguyen, Nickolas Falkner, Rhys Bowden, and Matthew Roughan. The internet topology zoo. *IEEE Journal on Selected Areas in Communications*, 29:1765 – 1775, October 2011.

[33] Alexander Krentsel, Nitika Saran, Bikash Koley, Subhasree Mandal, Ashok Narayanan, Sylvia Ratnasamy, Ali Al-Shabibi, Anees Shaikh, Rob Shakir, Ankit Singla, and Hakim Weatherspoon. A decentralized sdn architecture for the wan. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 938–953, New York, NY, USA, 2024. Association for Computing Machinery.

[34] Praveen Kumar, Yang Yuan, Chris Yu, Nate Foster, Robert Kleinberg, Petr Lapukhov, Chiun Lin Lim, and Robert Soulé. Semi-oblivious traffic engineering: The road not taken. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 157–170, 2018.

[35] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753. PMLR, 09–15 Jun 2019.

[36] Hongqiang Harry Liu, Srikanth Kandula, Ratul Mahajan, Ming Zhang, and David Gelernter. Traffic engineering with forward fault correction. In *Proceedings of ACM SIGCOMM*, pages 527–538, 2014.

[37] Libin Liu, Li Chen, Hong Xu, and Hua Shao. Automated traffic engineering in sdwan: Beyond reinforcement learning. In *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 430–435, 2020.

[38] Ximeng Liu, Zhuoran Liu, Yingming Mao, Yatao Li, Shizhen Zhao, and Xinbing Wang. Geminet: Learning the duality-based topology-agnostic update operator for lightweight traffic engineering in changing topologies. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*, 2026. To appear.

[39] Ximeng Liu, Shizhen Zhao, Yong Cui, and Xinbing Wang. Figret: Fine-grained robustness-enhanced traffic engineering. ACM SIGCOMM '24, page 117–135, New York, NY, USA, 2024. Association for Computing Machinery.

[40] Zaoxing Liu, Hun Namkung, Georgios Nikolaidis, Jeongkeun Lee, Changhoon Kim, Xin Jin, Vladimir Braverman, Minlan Yu, and Vyas Sekar. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches. In *30th USENIX Security Symposium (USENIX Security 21)*, 2021.

[41] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

[42] Yarin Perry, Felipe Vieira Frujeri, Chaim Hoch, Srikanth Kandula, Ishai Menache, Michael Schapira, and Aviv Tamar. DOTE: Rethinking (predictive) WAN traffic engineering. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1557–1581, Boston, MA, April 2023. USENIX Association.

[43] Michal Pióro and Deepankar Medhi. *Routing, Flow, and Capacity Design in Communication and Computer Networks*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2004.

[44] Mark Reitblatt, Nate Foster, Jennifer Rexford, Cole Schlesinger, and David Walker. Abstractions for network update. *SIGCOMM Comput. Commun. Rev.*, 42(4):323–334, August 2012.

[45] Arjun Singhvi, Nandita Dukkipati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyojeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang Wang, Masoud Moshref, Tao Ji, David Wetherall, and Amin Vahdat. Falcon: A reliable, low latency hardware transport. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25, page 248–263, New York, NY, USA, 2025. Association for Computing Machinery.

[46] Vineeth Sagar Thapeta, Komal Shinde, Mojtaba Malekpourshahraki, Darius Grassi, Balajee Vamanan, and Brent E. Stephens. Nimble: Scalable TCP-friendly programmable in-network rate-limiting. In *Proceedings of the ACM SIGCOMM Symposium on SDN Research (SOSR)*, 2021.

[47] Steve Uhlig, Bruno Quoitin, Jean Lepropre, and Simon Balon. Providing public intradomain traffic matrices to the research community. *SIGCOMM Comput. Commun. Rev.*, 36(1):83–86, jan 2006.

[48] Amin Vahdat and Nick McKeown. Fireside chat with Amin Vahdat (by Nick McKeown):Networking in the era of Warehouse Computing and AI. https://www.youtube.com/watch?v=qn7d_iK92TI.

[49] Asaf Valadarsky, Michael Schapira, Dafna Shahaf, and Aviv Tamar. Learning to route. In *Proceedings of the 16th ACM Workshop on Hot Topics in Networks*, HotNets '17, page 185–191, New York, NY, USA, 2017. Association for Computing Machinery.

[50] Erico Vanini, Rong Pan, Mohammad Alizadeh, Parvin Taheri, and Tom Edsall. Let it flow: Resilient asymmetric load balancing with flowlet switching. In *USENIX NSDI*, 2017.

[51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[52] Hao Wang, Haiyong Xie, Lili Qiu, Yang Richard Yang, Yin Zhang, and Albert Greenberg. COPE: Traffic engineering in dynamic networks. In *Proceedings of ACM SIGCOMM*, pages 99–110, 2006.

[53] Ye Wang, Hao Wang, Ajay Mahimkar, Richard Alimi, Yin Zhang, Lili Qiu, and Yang Richard Yang. R3: Resilient routing reconfiguration. In *Proceedings of ACM SIGCOMM*, pages 291–302, 2010.

[54] Yiting Xia, Ying Zhang, Zhizhen Zhong, Guanqing Yan, Chiun Lin Lim, Satyajeet Singh Ahuja, Soshant Bali, Alexander Nikolaidis, Kimia Ghobadi, and Manya Ghobadi. A social network under social distancing: Risk-Driven backbone management during COVID-19 and beyond. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 217–231. USENIX Association, April 2021.

[55] Zhiying Xu, Francis Y. Yan, Rachee Singh, Justin T. Chiu, Alexander M. Rush, and Minlan Yu. Teal: Learning-accelerated optimization of wan traffic engineering. In *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM SIGCOMM '23, page 378–393, New York, NY, USA, 2023. Association for Computing Machinery.

[56] Zhiyuan Xu, Jian Tang, Jingsong Meng, Weiyi Zhang, Yanzhi Wang, Chi Harold Liu, and Dejun Yang. Experience-driven networking: A deep reinforcement learning based approach. In *IEEE INFOCOM 2018 - IEEE Conference on Computer Communications*, page 1871–1879. IEEE Press, 2018.

[57] Manzil Zaheer, Satwik Kottur, Siamak Ravanbhakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J Smola. Deep sets. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 3394–3404, Red Hook, NY, USA, 2017. Curran Associates Inc.

[58] C. Zhang, Z. Ge, J. Kurose, Y. Liu, and D. Towsley. Optimal routing with multiple traffic matrices tradeoff between average and worst case performance. In *13TH IEEE International Conference on Network Protocols (ICNP'05)*, pages 10 pp.–224, 2005.

[59] Junjie Zhang, Minghao Ye, Zehua Guo, Chen-Yu Yen, and H. Jonathan Chao. Cfr-rl: Traffic engineering with reinforcement learning in sdn. *IEEE Journal on Selected Areas in Communications*, 38(10):2249–2259, 2020.

A Proof of Observation 1

Proof. Assume that the split ratio of d_{RQ} is $(\alpha, \bar{\alpha})$ along $(\overrightarrow{\text{RQ}}, \overrightarrow{\text{RSQ}})$ where $\overrightarrow{\text{RQ}}$ represents the direct link and $\overrightarrow{\text{RSQ}}$ is the tunnel via S. Similarly, d_{PQ} is divided using split ratios $(\beta, \bar{\beta})$ along $(\overrightarrow{\text{PQ}}, \overrightarrow{\text{PSQ}})$ respectively. Then, to satisfy the capacity constraints on $\overrightarrow{RQ}$ and $\overrightarrow{PQ}$, we must have $\alpha, \beta \leq \frac{1}{2}$. Moreover, the capacity constraints of $\overrightarrow{\text{RSQ}}$ and $\overrightarrow{\text{PSQ}}$ enforce $2\bar{\alpha} + \bar{\beta} \leq 1$ and $\bar{\alpha} + 2\bar{\beta} \leq 1$ respectively. By adding these constraints, we have $3(\bar{\alpha} + \bar{\beta}) \leq 2$. Then, the average traffic carried across the two scenarios is

$$\frac{3}{2}(\alpha + \bar{\alpha} + \bar{\beta} + \beta) \leq \frac{3}{2}\left(\frac{1}{2} + \frac{1}{2}\right) + 1 = \frac{5}{2},$$

where we have used $\alpha, \beta \leq \frac{1}{2}$ and $3(\bar{\alpha} + \bar{\beta}) \leq 2$. To see this is achievable, let $\alpha = \beta = \frac{1}{2}$ and $\bar{\alpha}, \bar{\beta} = \frac{1}{3}$. □

B Formulating Colt

$$\underset{w^n_{stk}, w^b_{stk}, T_{st}, d^{h,n}_{st}, d^{h,b}_{st}, g^h_{st}}{\text{maximize}} \quad \sum_{d^h \in \mathcal{D}} \sum_{s,t,k} \left(w^n_{stk} d^{h,n}_{st} + w^b_{stk} d^{h,b}_{st} \right) \tag{6a}$$

subject to

$$\sum_k w^n_{stk} \leq 1, \qquad \forall \langle s,t \rangle, \tag{6b}$$

$$\sum_k w^b_{stk} \leq 1, \qquad \forall \langle s,t \rangle, \tag{6c}$$

$$\sum_{s,t,k} \left(w^n_{stk} d^{h,n}_{st} + w^b_{stk} d^{h,b}_{st} \right) P_{ijstk} \leq C_{ij}, \forall \langle i,j \rangle, \forall h, \tag{6d}$$

$$d^{h,n}_{st} + d^{h,b}_{st} = d^h_{st}, \qquad \forall \langle s,t \rangle, \forall h, \tag{6e}$$

$$d^{h,n}_{st} \leq T_{st}, \qquad \forall \langle s,t \rangle, \forall h, \tag{6f}$$

$$d^{h,b}_{st} \leq M \cdot g^h_{st}, \qquad \forall \langle s,t \rangle, \forall h, \tag{6g}$$

$$T_{st} \geq d^h_{st} - M \cdot g^h_{st}, \qquad \forall \langle s,t \rangle, \forall h, \tag{6h}$$

$$d^{h,n}_{st} \geq T_{st} - M(1 - g^h_{st}), \qquad \forall \langle s,t \rangle, \forall h, \tag{6i}$$

$$d^{h,n}_{st}, d^{h,b}_{st}, w^n_{stk}, w^b_{stk}, T_{st} \geq 0, \qquad \forall s,t,k,h, \tag{6j}$$

$$g^h_{st} \in \{0,1\}, \qquad \forall \langle s,t \rangle, \forall h \tag{6k}$$

Here, M is a constant at least as large as any demand, i.e., $M \geq \max_h d^h_{st}$.

Equation (6) shows the mathematical formulation for obtaining Colt's piecewise-linear adaptive policy. The approach decomposes every demand into two components

$$d^{h,n}_{st} = \min(d^h_{st}, T_{st}) \tag{7}$$

$$d^{h,b}_{st} = \max(0, d^h_{st} - T_{st}) \tag{8}$$

and then optimizes two sets of split ratios, w^n_{stk} and w^b_{stk}, one per demand component. The objective is to maximize the average carried flow of different traffic scenarios $d^h_{st} \in \mathcal{D}$, as shown in Equation (6a), while respecting the capacity constraints of the links by each individual traffic scenario d^h_{st}, as shown in Equation (6d). Equations (6e)-(6i) show the required constraints to split each demand d^h_{st} into two components $d^{h,n}_{st}$ and $d^{h,b}_{st}$ while conserving the demand value as shown by Constraint (6e). Constraints (6f)–(6i) implement the threshold gating logic using a Big-M formulation. Constraint (6f) limits the baseline component, $d^{h,n}_{st}$, to the threshold T_{st}, conforming with Equation (7), while (6g) ensures the bursty component, $d^{h,b}_{st}$, is only active when the binary indicator, g^h_{st}, is equal to one. Constraints (6h) and (6i) force the binary indicator, g^h_{st}, to 1 when demand exceeds the threshold and 0 otherwise, ensuring the traffic is partitioned correctly based on the learned value of T_{st}. If the binary indicator, g^h_{st} is triggered ($d^h_{st} > T_{st}$), then $d^{h,b}_{st}$ is ensured to conform with Equation (8).

At the end, we highlight that this formulation is non-convex due to the $(w^n_{stk} \cdot d^{h,n}_{st}, w^b_{stk} \cdot d^{h,b}_{st})$ terms, and it also contains binary variables, making it intractable to solve.

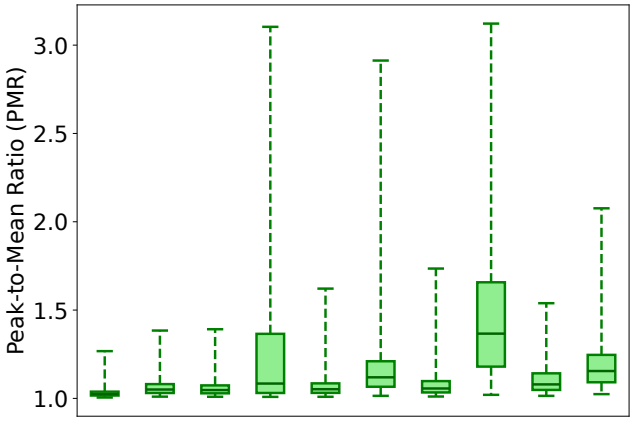


Figure 19: The distribution of Peak-to-Mean Ratio (PMR) for the ten most active pairs in GEANT across time. Whiskers indicate the 1st and 99th percentile. We assumed each TE cycle consists of five TMs.

C Investigating Correlated Bursts

Spearman correlation over the full dataset. While Figure 5 shows a representative 10-minute window, we here present the full-dataset study, covering all non-overlapping 10-minute windows. Figure 20 shows the distribution of Spearman correlations across windows for all combinations of the top-10 (highest-traffic-volume) pairs. Correlations are generally weak: only three combinations, (1,2), (3,5), and (6,9), show high median correlations (0.75, 0.76, and 0.71, respectively), while the medians of all other combinations are below 0.4, consistent with the representative window in Figure 5.

Multi-pair co-bursts. Spearman correlation only captures

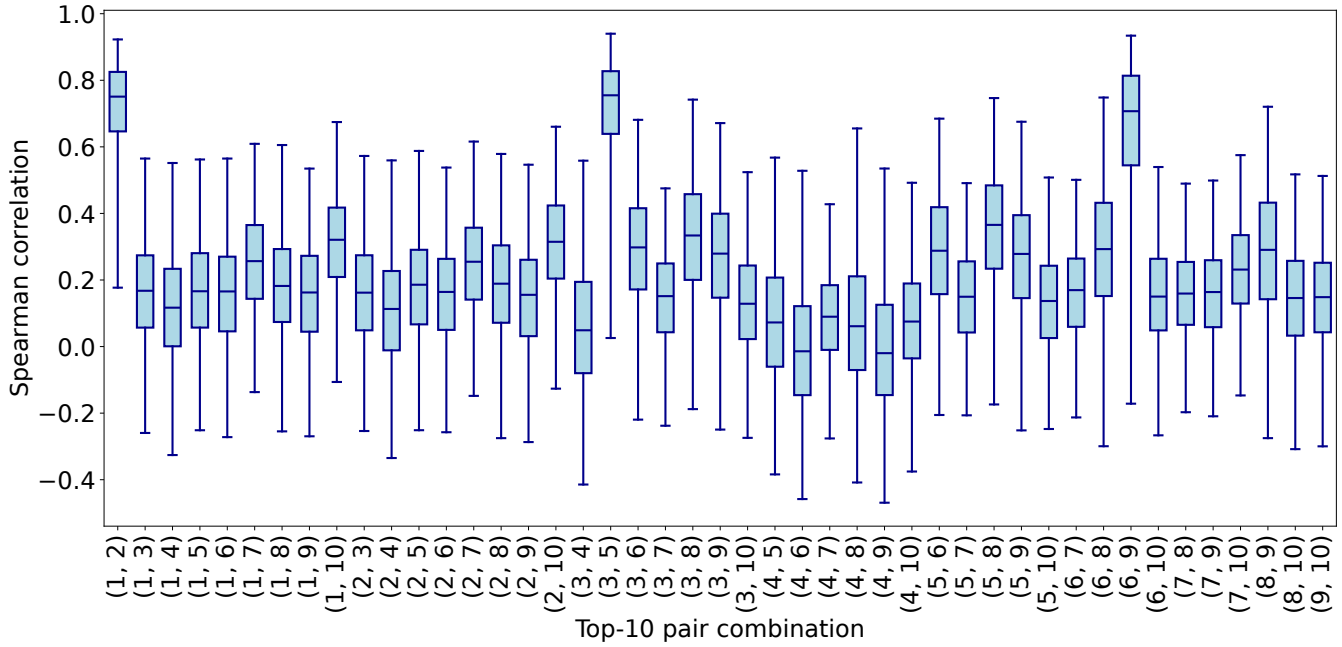


Figure 20: Distribution of pairwise Spearman correlation between the top-10 pairs' demands, computed independently within each non-overlapping 10-minute (50 TM) window, for all 45 pair combinations. Each box shows the 1st-99th percentile range across windows.

correlations across two pairs. We next investigate whether multiple pairs burst at the same time, and the impact on Colt-Opt's performance. Specifically, we identify the top-100 pairs in terms of traffic volume across time, and split the dataset into non-overlapping 10-minute windows. For each window, we compute the mean demand of each pair, and then divide each pair's demand in every TM of the window by that pair's window mean, which yields the demand-to-mean ratio (DMR). If DMR $\geq$ q (threshold), then that demand is considered bursty. Otherwise, it is considered non-bursty. We then compute the average number of co-bursty pairs (top-100) per TM in a cycle (5 TMs), matching the objective Colt-Opt optimizes (the average carried flow over the 5 TMs of a cycle). Figure 21 presents the CDF across cycles for three different q's (1.25, 1.333, 1.5). The figure shows that the median TE-cycle averages a modest 9, 5, and 1.4 co-bursty pairs per TM at q=1.25, 1.333, and 1.5, respectively. Even the 90th percentile stays relatively moderate, at 15.4, 10.2, and 4.2.

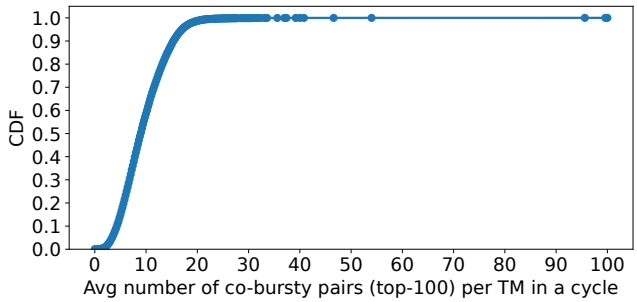
Impact of co-bursts while adjusting for traffic volume. §5.5 has shown that while Colt-Opt's performance degrades slightly with upto 20 co-bursty pairs the trend does not hold at higher co-bursty levels. Recall that traffic volume served as a confounding factor with more pairs classified bursty when overall traffic volumes are lower. We next separate the effect of co-bursts from that of traffic volume. We compute the total traffic volume of each TM (across all pairs) and the average across cycles. We split TE cycles into five quintiles based on the average total traffic volume. Within each quintile, traffic volumes are close (mean volumes differs by at most 2.2%) while the variation is about 35% between the bottom and top quintiles.

Figure 22 shows Colt's performance for each of the five traffic quintiles. Each curve corresponds to a quintile, and each point shows the mean NormFulFill for TE cycles in that quintile with a given number of co-bursty pairs. The results shows that Colt-Opt's mean NormFulFill consistently decreases with more co-bursts at every quintile. Further, the impact is greater at larger traffic volumes. In the bottom quintile, the mean NormFulFill barely drops, from 0.9998 in the first co-bursty group ([0, 5)) to 0.9991 in the fourth co-bursty group ([15, 20)), whereas in the top quintile, it drops by 0.6 percentage points (from 0.9964 to 0.9904). However, even in the worst case (highest traffic volume and most co-bursty pairs), Colt still achieves a mean NormFulFill of 0.9904, indicating the performance degradation is acceptable.

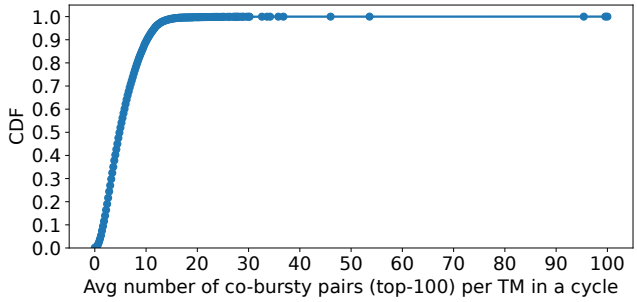
D Prediction Error and Impact

In this section, we quantify prediction error in PrivWAN and its impact on performance, and also consider the impact of a linear regression predictor.

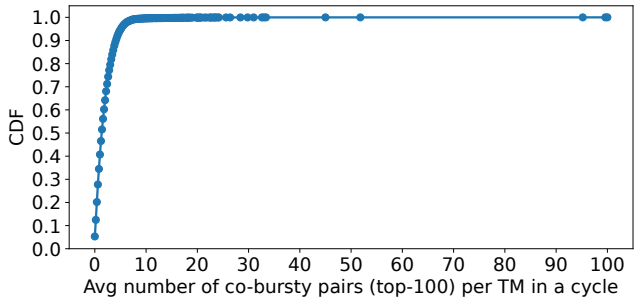
Measuring prediction error. To quantify prediction error, we fit a linear regression per TM pair that takes the average demand of that pair in each of the last 12 TE cycles and predicts the average demand of that pair in the next cycle. We



(a) q = 1.25



(b) q = 1.333



(c) q = 1.5

Figure 21: CDF of the average number of co-bursty pairs (top-100) per TM in a TE-cycle.

refit the model frequently, on a rolling window, so the model coefficients track traffic drift.

From Table 2 recall that $\mathcal{M}$ denotes the set of TMs in a future cycle, with each TM being denoted by $\mathbf{m}^f$. We augment the notation with (i) $\mathbf{m}^{avg}$: the true average demand over all $\mathbf{m}^f \in \mathcal{M}$, and ii) $\hat{\mathbf{m}}^{avg}$: the predicted demand. Note that $\mathbf{m}^{avg}$, $\hat{\mathbf{m}}^{avg}$, and $\mathbf{m}^f$ correspond to full matrices.

We measure prediction error using NRMSE (root mean square error, normalized as described in the next paragraph). We investigate two errors: i) NRMSE($\hat{\mathbf{m}}^{avg}$, $\mathbf{m}^{avg}$), the error in predicting the average TM in each cycle; ii) NRMSE($\hat{\mathbf{m}}^{avg}$, $\mathbf{m}^f$), the error in predicting each of the finer-timescale TMs within a cycle. We also compute NRMSE($\mathbf{m}^{avg}$, $\mathbf{m}^f$), which measures intra-cycle variation by computing how each TM within a cycle deviates from $\mathbf{m}^{avg}$. Figure 23 illustrates the three terms geometrically for a TE cycle of three matrices with two demands each.

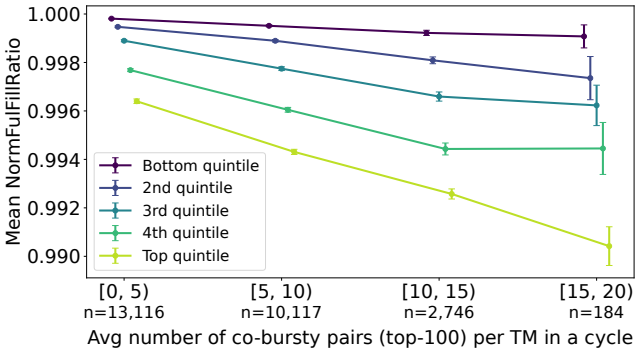


Figure 22: Performance of Colt-Opt as the number of co-bursty pairs increases. Each curve corresponds to TE cycles in a particular quintile by average traffic volume in a TE-cycle. Error bars show 95% confidence intervals of the mean.

We compute NRMSE as follows:

$$NRMSE(x,y) = \frac{\sqrt{\sum_{s,t}(x_{st}-y_{st})^2}}{\sqrt{\sum_{s,t}(m^{avg}_{st})^2}} \times 100\% \qquad (9)$$

Here x and y represent predicted and target matrices, respectively. Note that we normalize by $\mathbf{m}^{avg}$ to ensure consistent normalization for all three terms above.

Insights from prediction error. Figure 24 depicts the three NRMSEs across more than 21K TE-cycles. First, when predicting the average TM, the median NRMSE is 9.3% and 90th percentile NRMSE is 12.7%. Second, the error in predicting individual TMs is much higher with a median NRMSE of 18.8% and 90th percentile NRMSE of 31.3%. Third, intra-cycle variation is significant with median NRMSE of 16.1% and 90th percentile of 28.6%. Overall, the results suggest that while there is noticeable error when predicting the average matrix (the typical error that coarse-grained TE schemes focus on), the error is even more significant when individual TMs within a cycle are considered which motivates Colt's approach.

Can alternate predictors improve NonAdap-Opt?. §5 shows Colt and its optimization variant Colt-Opt outperform NonAdap-Opt, when both schemes design for the TMs in the previous cycle. We next consider whether the performance of NonAdap-Opt can be improved with alternate predictors.

The linear regression predictor that predicts the mean TM of each cycle as described above (which we refer to as LinReg-Mean) only provides a single matrix. This does not allow NonAdap-Opt the ability to optimize across multiple matrices. Hence, we also considered an alternate approach LinReg-5-TMs. Here, all TMs of the previous five TE cycles (25 TMs in all) are considered, and five separate linear regression models are built, each of which predicts an individual matrix of the next cycle. We then evaluate NonAdap-Opt when provided with the set of 5 predicted matrices.

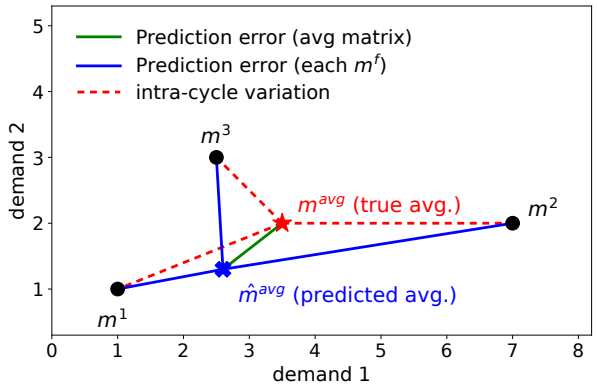


Figure 23: Geometric view of the errors associated with prediction for a TE cycle of three matrices with two demands each.

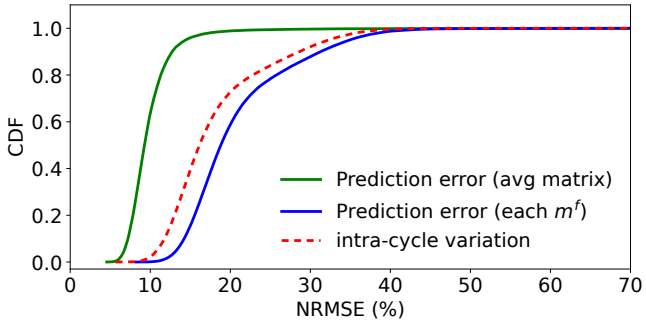


Figure 24: Distribution of the three NRMSEs across TE cycles in PrivWAN.

Figure 25 shows NonAdap-Opt's performance with LinReg-Mean, LinReg-5-TMs, and Prev-Cycle-5-TMs (the default scheme where the TMs of the previous cycle are provided). The results show that NonAdap-Opt performs similarly with LinReg-5-TMs and Prev-Cycle-5-TMs. The performance with LinReg-Mean is worse – this is expected since NonAdap-Opt does not benefit from optimizing for multiple matrices. This also indicates the importance of considering traffic variation within a TE cycle.

E Colt's Resilience to Prediction Error

To further assess Colt's resilience to prediction error, we apply a controlled perturbation to the future traffic matrices. Concretely, we modify each m^f_{st} in the set of future traffic matrices $\mathcal{M}$ as following:

$$\widetilde{m}^f_{st} = \max(0,\ d^{avg}_{st} + \beta \times (m^f_{st} - d^{avg}_{st})) \qquad (10)$$

where d^{avg}_{st} is the average of predicted matrices (TMs of the previous TE-cycle). Larger β implies larger perturbation, with $\beta = 1$ leaves the future traffic matrices unchanged. Figure 26

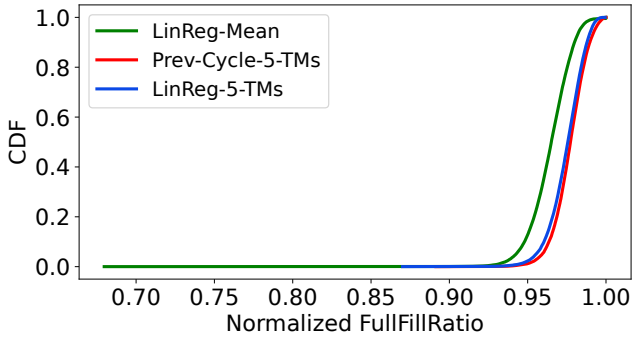


Figure 25: Normalized FulfillRatio of NonAdap-Opt on PrivWAN under three predictors. The performance with Prev-Cycle-5-TMs and LinReg-5-TMs, which both supply five matrices is nearly indistinguishable, while the performance with LinReg-Mean, which supplies only the predicted cycle average TM is consistently worse.

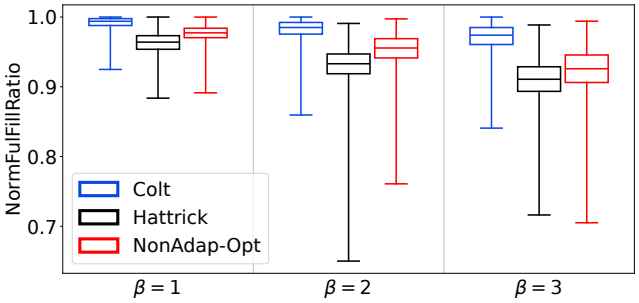


Figure 26: NormFulFill of Colt, Hattrick and NonAdap-Opt on PrivWAN while perturbing future traffic matrices to simulate larger prediction error. larger β implies larger perturbation with $\beta = 1$ adding no perturbation.

shows that Colt consistently achieves the best performance and degrades more gracefully as the perturbation strength increases. Increasing β from 1 to 3 lowers the median NormFulFill of Colt only from 0.994 to 0.974, compared with 0.964 to 0.911 for Hattrick and 0.977 to 0.926 for NonAdap-Opt, demonstrating Colt's greater resilience to prediction error.

F Impact of Intra-TE-Cycle Variation

To show the impact of intra-TE-cycle variation on performance, we compare NonAdap-Opt and Colt-Opt on PrivWAN, assuming optimal prediction of demands. Recall that under optimal prediction, NonAdap-Opt is the best non-adaptive scheme. Figure 27 shows the performance across different TE-cycle lengths. For the default 1-minute TE cycle of PrivWAN, the median NormFulFill of Colt-Opt and NonAdap-Opt are close (0.999 vs. 0.992), but the gap widens toward the tail. At three nines (the 0.1%ile), Colt-Opt still achieves 0.986, whereas NonAdap-Opt falls to 0.953, which translates into better SLO compliance. Longer TE cycles amplify this gap,

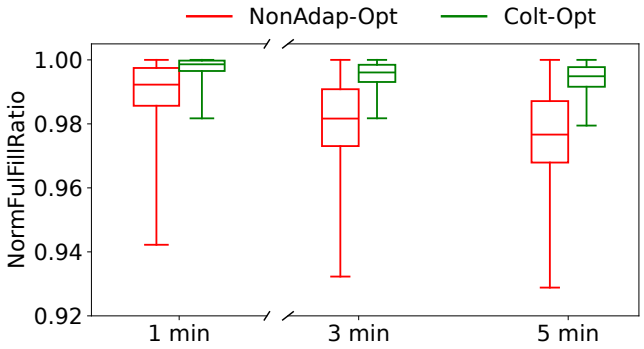


Figure 27: Comparing Colt-Opt and NonAdap-Opt on PrivWAN assuming optimal prediction for different TE-cycle lengths indicated on the x-axis.

since a single fixed routing must then accommodate the bursts of 15 (3 minutes) or 25 (5 minutes) TMs rather than 5. With 3-minute cycles, the median and three nines of NonAdap-Opt drop to 0.982 and 0.939, whereas Colt-Opt achieves 0.996 and 0.984, respectively. With 5-minute cycles, NonAdap-Opt falls further to 0.977 and 0.935, while Colt-Opt remains at 0.995 and 0.983. Thus, the gap becomes visible even at the median, as Colt-Opt's piecewise-linear policy absorbs the additional bursts within each cycle.